TEKNOLISTA / GUIA DE ENGENHARIA

Desenvolvimento de software com agentes de IA

Vibe coding, SDD e OpenSpec com as skills da Teknolista

Da primeira hipótese ao software instalado ou em produção. Laboratórios de Android, desktop, web, APIs HTTP, servidores MCP e workers.

COMECE COM UMA JORNADA

Escolha o fluxo e transforme intenção em comportamento verificável.

EXECUTE NO ALVO

Teste código, dados, compatibilidade e experiência de uso.

ENTREGUE COM EVIDÊNCIA

Verifique a versão instalada ou em serviço e planeje a recuperação.

ABRIR SUMÁRIO

EDIÇÃO CONSOLIDADA REVISADA / 11 NOV 2026 107 PÁGINAS

SUMÁRIO

Encontre seu ponto de partida

Clique numa linha ou use os marcadores do PDF. O cabeçalho TEKNOLISTA retorna a este sumário.

Método comum

Escolher entre edição direta, protótipo e SDD	03
Preparar o projeto, instalar e acionar skills	05
Laboratório A · OpenSpec da primeira change ao archive	08
Laboratório B · evolução de API e isolamento de tenants	20
Produção · pipeline, VPS, AWS, Azure, Google Cloud e Cloudflare	22
Edição direta, vibe coding e promoção de protótipos	34

Controles e laboratórios por plataforma

Primeira execução, paridade e release · lições do Castas	38
Segurança, verificação e entrega pelo Git	43
Android · preparar, especificar e implementar Notas de Bolso	45
Android · testes, revisão e evidência	63
Android · upgrade, APK, AAB e Play	70
Preparar os projetos web, desktop e serviços	78
Web · portal autenticado, navegador e preview	79
Desktop · arquivos locais, instalador e upgrade	81
API nova · da primeira transação ao ambiente alvo	83
MCP · descoberta, transporte e isolamento	84
Workers e integração entre versões de clientes	87
Outros alvos · CLI, dados, iOS e embarcados	89
Laboratório executável · da change ao CLI verificado	90

Adoção e consulta

Passagem de contexto, piloto e retrospectiva	94
Catálogo das 22 skills e checklist	100
Evidências desta edição	102
Diagnóstico e glossário Android	104
Fontes da consolidação	107

01 / DECISÃO

Escolha o fluxo pelo impacto

A quantidade de especificação depende do que pode dar errado e do custo de descobrir isso tarde.

Modo	Use quando	Entrega mínima
Direto	Objetivo claro, efeito local, contrato conhecido e fácil reversão.	Alteração focada, diff e checagem relevante.
Exploratório	Uma hipótese de uso ou viabilidade precisa ser testada.	Protótipo isolado, observação, limite e decisão seguinte.
SDD	Comportamento novo relevante, várias fronteiras, risco material ou política que exige especificação.	Contrato acordado com cenários, plano, código e evidência.

Faça três perguntas separadas

01 **O que pode ser delegado?** Deixe o agente resolver escolhas técnicas rotineiras dentro do escopo. Preserve as decisões de negócio e os limites de ação definidos pelo responsável.

02 **Onde a intenção precisa ficar?** Um pedido ou ticket pode bastar para uma correção. Um contrato durável facilita manutenção, revisão e retomada de funcionalidades.

03 **O que prova a mudança?** Escolha a verificação pela consequência. Uma cor pode exigir contraste. Uma condição de autorização exige casos permitidos e negados, inclusive efeitos no estado.

REAVALIE DURANTE O TRABALHO

Uma correção de texto que revela um problema de permissões vira outra decisão. Preserve a edição autorizada quando independente. Estabeleça o contrato da mudança de permissão antes de tocar no comportamento afetado.

Neste guia, **vibe coding** é exploração conversacional com baixo custo de erro. Para uma edição real e pequena, use um prompt curto acompanhado da checagem que protege o contrato existente.

02 / ARQUITETURA DO FLUXO

Um dono para cada regra

PentaPRO organiza o contexto do produto. OpenSpec registra o comportamento detalhado das capacidades e as mudanças que o alteram.

Registro	O que governa
docs/product.md	Problema, público, resultado, escopo e regras globais. Vincula capacidades e critérios.
docs/proof.md	Plano de verificação, relação entre requisito e cenário, resultados por revisão e limites.
docs/project.md	Arquitetura global, dados, contratos técnicos, decisões, riscos e dependências.
docs/production.md	Ambientes, instalação, operação, migrações, monitoramento, recuperação e suporte.
docs/process.md	Prontidão, autoridade, responsáveis, histórico, próxima ação e apontador para a change.
docs/ui-design.md	Quando a especificação de interface está em escopo, telas, fluxos, estados e acessibilidade.
openspec/specs/	Comportamento consolidado por capacidade no snapshot da especificação.
openspec/changes/<id>/	Proposta, delta, design e tarefas da mudança no schema spec-driven.

Se o projeto já tem um ticket, OpenAPI, modelo de estados ou teste que governa parte do contrato, vincule a fonte exata. Preserve a decisão e o escopo em seu registro oficial. Os pilares podem funcionar como índice, inclusive em projetos com formato próprio.

SEPARE OS ESTADOS

Rascunho, requisito aceito, código implementado, verificação, integração e implantação respondem a perguntas diferentes. Um check deve apontar para a revisão em que rodou.

03 / INSTALAÇÃO

Prepare a raiz para OpenSpec

Use um projeto descartável nos laboratórios. Abra no Codex a raiz do software, onde ficam seus comandos, contratos e instruções. OpenSpec é opcional nos fluxos direto e exploratório.

Projeto novo com a versão fixada no guia

```
node --version
npm install -g @fission-ai/openspec@1.17.0
openspec --version
openspec init . --tools codex --profile core --language English
openspec list
openspec list --specs
```

O comando npm instala a CLI no ambiente do usuário. O pacote OpenSpec pede Node.js 22.12 ou posterior. O Skills CLI 1.7.6 pede Node.js 22.20 ou posterior. Escolha uma linha suportada que atenda aos dois requisitos. Se preferir um instalador independente, consulte a documentação e registre a versão que será usada pela equipe.

Projeto existente

```
git status --short --branch
openspec --version
openspec list
openspec update --dry-run --json
```

O último comando é da versão 1.17.0. Confira ajuda e comportamento em outras versões. Leia a configuração existente, resolva a raiz da especificação, o perfil, o schema e a change ativa. Atualize a integração quando essa ação fizer parte do trabalho. Revise os arquivos gerados e preserve as regras locais em AGENTS.md ou na configuração do projeto.

Confira o resultado

Use openspec ... no terminal e as menções $openspec-... no chat. Confira os nomes realmente gerados em .agents/skills/. O perfil core fornece o ciclo principal. Recursos opcionais exigem o perfil correspondente. Trate uma integração ausente como questão de instalação ou descoberta.

OpenSpec · instalação e alternativas

OpenSpec 1.17.0 · release e atualização

04 / SKILLS NO CODEX

Instale e confirme a seleção

As skills geradas pelo OpenSpec e as skills Teknolista cumprem papéis diferentes. Faça a instalação na raiz do produto. Comece pelas skills que sua tarefa requer.

Instalação reproduzível com checkout local

```
SKILLS_REPO=/caminho/para/teknolista/skills
git -C "$SKILLS_REPO" rev-parse HEAD
git -C "$SKILLS_REPO" status --short
npx skills@1.7.6 add "$SKILLS_REPO" --list
npx skills@1.7.6 add "$SKILLS_REPO" --skill penta-pro -a codex --copy
# Para instalar as 22 de uma vez:
npx skills@1.7.6 add "$SKILLS_REPO" --skill '*' -a codex --copy -y
```

Troque a origem por teknolista/skills para usar a coleção publicada. Confira a versão do pacote desejado na origem. Para repetir exatamente um piloto, fixe a revisão de origem e a versão do instalador. Uma cópia local com alterações também exige registrar esse estado.

Instalação manual

Copie cada pasta completa para .agents/skills/ na raiz do projeto. Preserve SKILL.md, licença, referências, modelos e scripts. Verifique que as skills openspec-* continuam disponíveis. Reabra a sessão se o catálogo estiver antigo. Em outro host, use a pasta e a sintaxe documentadas pelo host.

Descoberta, seleção e aplicação

```
$penta-pro
Identify the governing spec, the next increment and missing evidence.
Read only. Report the skill version and the records you inspected.
```

Veja se a skill aparece no seletor, confirme o identificador e leia o resultado. Um contrato, uma ação ou uma checagem pertinente mostra como o procedimento foi aplicado. Em homologação hospedada, use $production-operations e siga o roteiro da página 22.

Teknolista · instalação, ativação e hosts

05 / POLÍTICA

Defina o mínimo em AGENTS.md

Adapte o AGENTS-template.md à raiz do software. Preencha comandos, donos e limites reais antes de usar os exemplos como política de equipe.

AGENTS.MD / EXEMPLO

```
- Use the local contract and focused checks for reversible edits.
- Use OpenSpec for material behavior changes and active tasks.
- Link project-wide decisions and verification in the pillars.
- Before coding, resolve material decisions for the scoped work.
- If whole-scope readiness is required, review all of that scope.
- Keep test, merge, release and external-action authority explicit.
- On contract drift, pause dependent work and consult the owner.
```

OPENSPEC/CONFIG.YAML / EXEMPLO PARA INTEGRAR AO EXISTENTE

```
schema: spec-driven
context: |
  Read AGENTS.md and the relevant project contracts.
  Use English for specification and engineering artifacts.
rules:
  proposal:
    - State the outcome, scope, and exclusions.
  specs:
    - State observable behavior and failure cases.
  tasks:
    - Link the scenario and name the evidence needed.
```

Preencha o comando de teste, lint, build, ambientes e critérios de integração. Defina quem decide exceções e quais checks são obrigatórios. Em repositórios com política diferente por diretório, confira as instruções aplicáveis ao arquivo.

VERSIONE O CONTEXTO

No fluxo local padrão, mantenha **openspec/** no Git, incluindo configuração, specs, changes e arquivo histórico conforme a política. Para integração e skills geradas que a equipe optar por ignorar, documente a instalação reproduzível. Use CI para impor checks executáveis.

06 / LABORATÓRIO A

Comece com um resultado de uso

Notas de Bolso é um app Android nativo didático. O resultado é salvar uma nota sem rede e reabri-la depois de reiniciar.

Decisão	Escopo deste exercício
Usuário e jornada	Pessoa que quer capturar uma nota sem login. Lista → editor → salvar → reiniciar → reabrir.
Dentro	Criar, editar e persistir uma nota de texto localmente. Tratar erro de gravação e entrada vazia.
Fora	Contas, rede, sincronização, compartilhamento e recuperação de rascunho após reinício.
Política	Gravação bem-sucedida só após commit local. Em falha, manter o valor persistido e preservar o rascunho no editor ativo.

CHAT / CRIAR E REVISAR A PRIMEIRA ESPECIFICAÇÃO

```
$penta-pro
Create the five pillar docs for Notas de Bolso.
Use the agreed offline note journey and exclusions.
Use OpenSpec for the detailed capability and active tasks.
Ask about material gaps, mark proposals and pending decisions,
and review the saved draft. Specification only.
```

Em produto novo, o padrão da skill é criar os cinco documentos. Em produto existente, leia primeiro o contrato e a estrutura atuais. Vincule os registros oficiais e preserve os IDs. Gere `docs/ui-design.md` se detalhar a interface fizer parte do pedido.

CHECAGEM DE LEITURA

Peça a alguém para reconstruir a jornada só com os documentos. Se o texto diz apenas “salvar com sucesso”, a reabertura depois de reiniciar e a recuperação após falha ainda precisam ser explicitadas.

07 / PRONTIDÃO

Especifique antes de codificar

Escolha o escopo que precisa estar pronto. Use a próxima fatia como padrão, ou toda a funcionalidade delimitada quando o pedido ou a política exigir.

Estado	O que o agente pode afirmar
Confirmado	O responsável ou registro aplicável definiu a regra. A origem fica rastreável.
Proposto	A decisão é uma recomendação. Pode incluir opções, efeitos e uma proposta de verificação.
Pendente	Existe uma pergunta ou evidência em aberto. Nomeie dono, trabalho dependente e quando resolver.
Pronto	O escopo acordado tem contrato, cenários, dependências e checks viáveis. Execução continua sujeita à autoridade real.

Exemplo de bloqueio que muda o desenho

Se a recuperação do rascunho após o encerramento do processo estiver em escopo e ainda for indefinida, a escolha muda armazenamento, privacidade e testes. Registre-a como decisão pendente. Neste exercício, o responsável decidiu preservar apenas no editor ativo, o que permite especificar o comportamento completo da fatia.

CHAT / AUTORIZAÇÃO COM CONDIÇÃO EXPLÍCITA

```
$penta-pro
Specify the entire selected change before any implementation.
Use existing evidence and leave unresolved policy choices pending.
Save the artifacts and review readiness for the full scope.
When all required decisions are settled, implement within scope.
Keep implementation, integration and release gates distinct.
```

QUANDO O TRABALHO PODE SEGUIR

Se o pedido combinado já autoriza código, prossiga após cumprir os gates aplicáveis e resolver o escopo requerido. Se o pedido é só de especificação, encerre com os artefatos e o próximo passo.

08 / OPENSPEC CORE

Abra a change e confira os arquivos

Use o schema spec-driven do exemplo. Um schema personalizado pode alterar nomes, dependências e ordem dos artefatos.

CHAT / PROPOSTA

```
$openspec-propose notes-create-edit
Read the five pillars and the confirmed offline note decisions.
Draft the proposal, notes delta, design and tasks.
Cover save, edit, restart, write failure and empty input.
Exclude accounts, sync and sharing. Planning only.
```

ÁRVORE ESPERADA DA MUDANÇA

```
openspec/changes/notes-create-edit/
  proposal.md
  specs/notes/spec.md
  design.md
  tasks.md
```

Arquivo	O que procurar ao revisar
proposal.md	Por que, para quem, escopo, exclusões e capacidades afetadas.
specs/notes/spec.md	Requisitos e cenários observáveis. Cada delta tem uma relação explícita com a spec consolidada.
design.md	Fronteiras, decisões, alternativas e condições de falha ou recuperação.
tasks.md	Fatias dependentes, resultado observável, check e condição de conclusão.

TERMINAL / LEIA ESTADO E VALIDE FORMATO

```
openspec status --change notes-create-edit
openspec show notes-create-edit --type change
openspec validate notes-create-edit --type change --strict
```

Status e validação medem artefatos e formato. Reconstrua o comportamento esperado e confronte o plano com as decisões do projeto antes de aplicar.

09 / DELTA 1 DE 2

Escreva cenários observáveis

Copie as duas partes do delta na ordem para openspec/changes/notes-create-edit/specs/notes/spec.md. Purpose descreve a capacidade que será criada.

DELTA / PERSISTÊNCIA E EDIÇÃO

```
## Purpose
Support offline note capture and retrieval on one device.

## ADDED Requirements

### Requirement: Durable local note
The app SHALL persist a non-empty note without network access.
It SHALL show success only after the local write commits.

#### Scenario: Create and reopen offline
- **WHEN** a user saves "Buy milk" with networking disabled
- **THEN** the note appears in the list and can be reopened
- **AND** the same text is available after app restart

#### Scenario: Edit an existing note
- **WHEN** the user replaces "Buy milk" with "Buy bread"
- **THEN** the existing note contains "Buy bread" after restart
- **AND** no duplicate note exists
```

Ligue a promessa à prova

“Persistir” é uma promessa sobre o estado durável. O cenário verifica a nota pelo caminho do usuário depois de reabrir o app. Um teste que só observa o estado da tela pode passar enquanto o banco continua vazio.

Camada	Evidência que fornece
Unidade	Validação de conteúdo e transições de estado em condições controladas.
Integração	Gravação e leitura reais na fronteira de armazenamento escolhida.
Dispositivo	Jornada real em Android, incluindo reabertura do processo para a persistência prometida.

ESCREVA PARA A FRONTEIRA CERTA

Use exemplos com entradas, ações e resultados. Tipos, schemas e testes podem formalizar partes do contrato. Preserve no registro de origem as razões e restrições que esses artefatos não expressam.

10 / DELTA 2 DE 2

Defina falha e recuperação

Inclua o caso de erro com efeito verificável. A evidência precisa observar o que foi preservado e o que o usuário consegue fazer em seguida.

DELTA / CONTINUAÇÃO SEM REPETIR O CABEÇALHO ADDED

```
### Requirement: Recoverable save failure
The app SHALL preserve the last committed note on write failure.
It SHALL keep the edited draft in the active editor for retry.

#### Scenario: Storage write fails during edit
- **GIVEN** an existing note "Buy milk" and a draft "Buy bread"
- **WHEN** the repository reports that saving failed
- **THEN** the editor shows an error and keeps "Buy bread"
- **AND** the last committed note remains "Buy milk"
- **AND** the user can retry without retyping

#### Scenario: Empty note is rejected
- **WHEN** the user attempts to save whitespace-only text
- **THEN** the editor explains the validation error
- **AND** no note is created or overwritten
```

Escolha a operação adequada

Seção do delta	Uso
ADDED	Introduzir requisito que ainda não existe na capacidade.
MODIFIED	Substituir a definição do requisito existente. Use o título correspondente e inclua os cenários que continuam válidos.
REMOVED	Retirar um requisito segundo a política de compatibilidade. Registre motivo e migração quando aplicável.
RENAMED	Mudar o título preservando identidade e significado. Confira os vínculos de consumidores e outras changes.

Inclua um Purpose descritivo ao criar uma capacidade. Para alterações em requisitos consolidados, compare o delta com a spec de origem e com changes que tocam a mesma capacidade.

11 / DESIGN E TAREFAS

Planeje uma fatia de ponta a ponta

Comece pela jornada mínima que verifica a promessa mais importante através do ponto de entrada real.

Fronteira	Escolha deste exemplo
Editor	Captura texto e exibe estado de salvamento. Mantém rascunho durante falha no editor ativo.
Repositório	Controla a operação durável e reporta sucesso ou falha ao chamador.
Armazenamento	Nota com identidade estável. Atualizar a existente preserva o ID e evita duplicação.
Evidência	Asserção de persistência e estado visível. Falha injetada com o dado anterior preservado.

TASKS.MD / FATIA E CHECAGENS

```
## 1. Local note journey
- [ ] 1.1 Persist and reopen a note offline after restart.
- [ ] 1.2 Edit the same note and verify the stored value.
- [ ] 1.3 Exercise failed save, retry and empty input.
- [ ] 1.4 Record checks and coverage for the verified revision.
```

Defina a superfície da interface

Para um fluxo novo, use `$ui-design` quando for preciso especificar o editor. Documente estados de edição, salvando, salvo, vazio e falha. Vincule a regra de persistência à capacidade do OpenSpec. Verifique foco, leitura de erros, teclado, entrada e navegação no ambiente da plataforma.

Respeite a stack

Se o projeto usa Compose e Room, os helpers de Android cobrem UI, repositório e testes reais. Koin, WorkManager e novos módulos entram quando a arquitetura ou a tarefa os pede. A fatia deste exemplo funciona sem job de sincronização.

12 / GATE DE IMPLEMENTAÇÃO

Revise o que está autorizado

Antes de aplicar, confira o contrato com o desenvolvedor ou com quem responde pela regra de produto.

01 **Confirme a fonte.** Identifique a raiz, revisão e change. Compare a spec consolidada com o delta aceito. Separe propostas e a implementação atual.

02 **Percorra a jornada.** Leia escopo, sucesso, falhas, efeitos, entradas e exclusões. Verifique as dependências e se o critério pode ser observado.

03 **Resolva a decisão que bloqueia.** Use exemplos que mostrem o efeito de cada opção. Registre a escolha no artefato que a governa e atualize os cenários afetados.

04 **Defina a verificação.** Descreva o teste e seu oráculo antes de implementar. Registre o ambiente e a evidência exigida. Mantenha checks futuros como planejados.

05 **Confirme autoridade.** Registre o que foi autorizado em PROCESS ou no tracker oficial. Aceite da spec, permissão de implementação, revisão e autorização de release são registros distintos.

CHAT / REVISÃO DE PRONTIDÃO, SÓ LEITURA

```
$penta-pro
Review the saved pillars and change notes-create-edit.
Reconstruct the user journey from the files.
Check scope, decisions, failure scenarios and feasible checks.
Report readiness, blockers and the next allowed action.
Read only.
```

EXEMPLO DE DECISÃO PRÁTICA

Se o emulador estiver ausente, o teste instrumentado permanece pendente. A política do projeto determina se essa falta impede iniciar código ou apenas concluir a entrega.

13 / EXECUÇÃO

Implemente o próximo passo

Use o OpenSpec para localizar e atualizar a tarefa. Use a skill de domínio para orientar a execução no código e Software Delivery para coordenar dependências.

CHAT / EXECUÇÃO DA CHANGE AUTORIZADA

```
$openspec-apply-change notes-create-edit
Use $software-delivery for dependent tasks.
Start with the accepted offline save and reopen journey.
Use $android-data-layer for the storage boundary.
Preserve the change scope and the existing app conventions.
Run the relevant checks and cite their evidence.
Pause affected work if the contract needs a new decision.
```

01 **Estabeleça a linha de base.** Leia código, chamadas e testes. Rode checks seguros quando precisarem distinguir uma falha antiga da que a change pode introduzir. Confira onde o comando grava ou acessa serviços.

02 **Implemente e verifique a menor fatia completa.** Passe pelo ponto de entrada real. Se a equipe exige teste primeiro, observe a falha pela razão correta antes da implementação. Preserve asserções existentes que protegem o contrato.

03 **Atualize os registros ao concluir.** Marque tarefas com evidência real. Escreva o check, resultado, ambiente e revisão em PROOF ou no registro oficial. Aponte em PROCESS para a próxima tarefa e para o que segue pendente.

SE SURGIR DIVERGÊNCIA

Localize requisito, consumidor e teste afetados. Se o código diverge da regra, corrija o código. Se o requisito é insuficiente ou inviável, confirme a decisão com o dono e atualize o delta, design e tarefas antes de seguir no trabalho dependente.

14 / PROOF

Colete evidência que pode falhar

Escolha testes que distinguem o comportamento correto de uma implementação plausível e errada.

Check para o exercício	O que observar
Validação e estado	Um teste com texto em branco e um texto aceito. Falha deve preservar estado e rascunho em uso.
Armazenamento	Criar, fechar e reabrir com o mesmo dado. Editar pelo mesmo ID e confirmar que há só uma nota.
Jornada Android	Rede desligada, entrada real, confirmação, reabertura e leitura. Registre dispositivo ou emulador e API.
Falha controlada	Injete falha na fronteira apropriada. Verifique ausência de falso sucesso e preservação do valor anterior.

TERMINAL / EXEMPLOS PARA UM PROJETO COM ESSAS TAREFAS GRADLE

```
./gradlew testDebugUnitTest
./gradlew connectedDebugAndroidTest
```

Consulte os nomes reais das tarefas e a variante. O segundo comando precisa de dispositivo ou emulador. Use `$android-unit-testing`, `$android-integration-testing` ou `$android-ui-testing` conforme a fronteira. Fora do Android nativo, use `$behavior-testing` quando os testes são o resultado principal ou há pedido de teste primeiro.

PROOF / REGISTRO ILUSTRATIVO, ANTES DA EXECUÇÃO

```
Check: NOTE-REOPEN
Expected: same note ID and text after offline restart
Command/environment: project device test, API to be recorded
Result: Unverified
Evidence/revision: pending execution
```

Quando executar, anote comando exato, resultado, revisão e cobertura. Diferencie observação manual, integração simulada e execução real. Rerode a parte afetada se código, configuração ou contrato mudar.

15 / REVISÃO E SEGURANÇA

Cheque o resultado antes de integrar

Revise o diff e seu contexto. Faça a avaliação independente que a política ou o risco da mudança exige.

CHAT / REVISÃO DO SNAPSHOT

```
$peer-review
Review this diff and its tests against notes-create-edit.
Trace persistence, failure recovery and existing callers.
Report supported findings, tested scope and missing evidence.
Review only.
```

CHAT / REVISÃO DE RISCO MATERIAL

```
$cybersec
Assess the changed data and trust boundaries in this snapshot.
Identify threat, control, verification and residual risk.
Use authorized local checks. Report scope and evidence.
```

Olhar	O que pedir
Código e testes	Problema com reprodução ou trajetória causal, impacto, condição e evidência. Analise asserções e contraexemplos.
Segurança	Ativos, entrada não confiável, autorização no ponto do efeito, isolamento, dados sensíveis, dependências e rollback.
Independência	Use nova sessão ou revisor independente quando requerido e disponível. Registre autorrevisão quando for o caso.
Correções	Confronte achados com o contrato. Corrija quando autorizado e reexecute os checks afetados na revisão resultante.

O PAPEL DO VERIFY

No perfil expandido, **$openspec-verify-change** pode ajudar a comparar implementação e artefatos, se gerado. Use os testes e a revisão previstos pela equipe também no perfil core.

16 / CONSOLIDAÇÃO

Sincronize e arquive com evidência

Arquive quando as tarefas e os gates aplicáveis estiverem concluídos. Siga a ordem de integração do seu projeto.

01 **Revise o delta final.** Compare o resultado verificado com o contrato e o task list. Confira sobreposição com mudanças na mesma capacidade. Rode a validação estrita.

02 **Escolha a operação.** Se quiser inspecionar o merge de specs separadamente, use $openspec-sync-specs no chat. A change permanece ativa. No caminho comum, use o arquivamento para concluir e reconciliar as specs.

03 **Confira a árvore resultante.** Revise a spec consolidada e a pasta de archive. Atualize os vínculos que apontavam para a change ativa. Preserve a revisão testada e o histórico.

CHAT / FLUXO GUIADO DE ARQUIVAMENTO

```
$openspec-archive-change notes-create-edit
Check tasks, test evidence and delta before archiving.
Confirm the intended sync and show the resulting paths.
```

TERMINAL / CAMINHO ALTERNATIVO E CONFERÊNCIA

```
openspec validate notes-create-edit --type change --strict
openspec archive notes-create-edit
openspec list --archived
openspec validate --specs --strict
openspec validate --archived
git status --short
```

Escolha um caminho de arquivamento. O comando de terminal pode atualizar specs e mover os artefatos da change. Em um repositório compartilhado, revise o diff da sincronização junto com o código, conforme a política de integração.

O QUE FOI ENCERRADO

O arquivo prova que a change foi arquivada na revisão em que a operação ocorreu. CI, merge, publicação, implantação e disponibilidade para usuários têm evidências próprias.

17 / ENTREGA

Passe do diff à operação

Git, implantação e liberação para usuários têm resultados e evidências próprios.

CHAT / PREPARAR A ENTREGA PARA REVISÃO

```
$git-delivery
Inspect the intended Git snapshot and its required checks.
Report the paths, verification and any blockers.
Prepare a commit and PR plan. Read only for this turn.
```

Ao executar Git, identifique as ações e o destino autorizados. Confira hooks e workflows que podem disparar publicação, alteração de infraestrutura ou deploy ao receber um push ou merge.

Etapa	Resultado e evidência
Integrar	Diff revisado, checks da revisão, conflitos resolvidos e política de merge atendida.
Preparar o release	Artefato por digest ou versão, alvo, configuração, migrações, critérios e recuperação. Use **Production Operations**.
Implantar e promover	Versão observada no ambiente, sinais e verificações reais. Amplie o tráfego quando os critérios acordados forem observados.
Encerrar	Relacione change, revisão, release e evidência. Registre limites, responsáveis e pendências de operação.

QUANDO O DEPLOY É PARTE DA CHANGE

Se o acordo de aceitação exige implantação ou verificação no ambiente, mantenha essa tarefa pendente até obter a evidência. Se o release for um trabalho separado, vincule o registro operacional à change e à revisão. Continue na página 22.

Use **$pr-quiz** quando alguém pedir um exercício de compreensão do diff. Mantenha o resultado junto ao aprendizado e aplique os gates de revisão do projeto.

18 / LABORATÓRIO B

Evolua uma API existente

Um cliente pode repetir uma submissão depois de perder a resposta. Hoje, a chave global deixa dois clientes no mesmo espaço de identidade. A change define a separação por tenant.

Antes de escrever o delta	Decisão para este caso
Baseline observada	Leia spec atual, handler, autorização, armazenamento e consumidores. Registre diferenças entre o que roda e o que foi aceito.
Efeito prometido	Cada tenant pode usar a mesma chave independentemente. Uma repetição igual no mesmo escopo retorna o mesmo recibo e não cria outro job.
Conflito	Mudança de payload sob a mesma chave retorna 409. A regra e a forma do erro são decisões de produto e compatibilidade.
Operação	Se já existem chaves globais ou clientes dependentes, planeje coexistência, migração, telemetria e recuperação antes de mover dados.

CHAT / PROPOSTA SOBRE A SPEC EXISTENTE

```
$openspec-propose tenant-receipts
Read the existing receipts spec and real API consumers.
Draft a MODIFIED delta for Job receipt replay.
Preserve supported callers and the normal success shape.
Ask about legacy keys, replay conflicts and migration.
Planning only.
```

No exemplo didático, o responsável confirmou o 409 e o título existente é **Job receipt replay**. Se o responsável ainda não decidiu a política de conflito e ela faz parte do escopo que deve estar pronto, salve a proposta como pendente antes de implementar.

UMA LINHA PODE MUDAR O RISCO

“Torne a chave global para simplificar” contradiz o contrato de isolamento. O agente deve apontar a regra e o efeito, preservando o trabalho que depende dessa decisão.

19 / DELTA DE API

Preserve cenários e efeitos

Delta em openspec/changes/tenant-receipts/specs/receipts/spec.md. A spec principal contém o requisito e o cenário Replay an existing key.

DELTA / REQUISITO EXISTENTE COM CENÁRIOS PRESERVADOS

```
## MODIFIED Requirements

### Requirement: Job receipt replay
The endpoint SHALL authorize the caller before receipt lookup.
It SHALL scope keys by authenticated tenant and operation.
An exact replay SHALL return the original receipt.
A changed payload under the same scope and key SHALL return
HTTP 409 with idempotency_conflict and create no job.

#### Scenario: Replay an existing key
- **WHEN** a tenant repeats an accepted submission unchanged
- **THEN** the original receipt is returned without a new job

#### Scenario: Independent tenant
- **WHEN** a different tenant uses the same key
- **THEN** it gets its own receipt and job

#### Scenario: Changed payload
- **WHEN** the same tenant repeats the key with a changed payload
- **THEN** the response is 409 with idempotency_conflict
- **AND** the original job remains the only job for that key

#### Scenario: Unauthenticated caller
- **WHEN** a request lacks an authenticated principal
- **THEN** the response is 401 and no lookup result is disclosed
- **AND** no job is created
```

O cenário existente mantém o título e ganha condições mais precisas. A validação estrita do exemplo rejeitou a primeira versão porque o MODIFIED omitia esse cenário. Confira os cenários preservados antes de arquivar.

Implemente e verifique na fronteira real

Use `$http-api` para o handler e o efeito durável. Leia chave, operação, tenant autenticado e equivalência do payload. Exercite duas chamadas concorrentes, a perda da resposta após gravar e um acesso negado. Compare status, corpo e contagem de jobs. Para efeitos externos, use ambiente isolado e o mecanismo de idempotência ou reconciliação do fornecedor.

PRODUÇÃO / 01

Leve a API do laboratório ao ar

O contrato de recibos da página 21 passa a ser o roteiro de verificação da versão implantada.

Registro	O que escrever ou vincular
OpenSpec	Em **tenant-receipts**, mantenha os cenários de replay, isolamento, 409 e negação. Vincule as tarefas de compatibilidade e release quando fizerem parte da change.
PROJECT e PRODUCTION	Registre a topologia e as dependências. Defina alvo, configuração, migração, sequência de rollout, operação e recuperação em PRODUCTION.
PROOF e PROCESS	Em PROOF, compare revisão esperada e observada, tráfego, efeito persistido e limites. Em PROCESS, vincule a autorização, o dono e o próximo gate.

CHAT / PLANEJAMENTO DO RELEASE

```
$production-operations
Read tenant-receipts and the current release procedures.
Plan the staging pipeline, rollout and recovery for this API.
Tie each check to the existing OpenSpec scenario.
List the exact target and access references needed.
Planning only.
```

Transforme a intenção em tarefas verificáveis

TASKS.MD / EXEMPLO

```
- [ ] Rehearse schema expansion with the previous writer active.
- [ ] Deploy the pinned image to the approved staging target.
- [ ] Check replay, cross-tenant separation, 409 and 401.
- [ ] Record the running revision and persisted job counts in PROOF.
```

Se a operação tiver escopo próprio, use um registro de release ligado à change. Uma nova change de infraestrutura só precisa de delta de capacidade quando o schema escolhido o exigir ou houver comportamento governado a alterar. Use o schema e a política do projeto.

ESTADOS DO RELEASE

Registre preparado, aplicado, em serviço e verificado com a evidência correspondente. Um deploy que criou a revisão, mas ficou com 0% de tráfego padrão, ainda requer a verificação da promoção.

PRODUÇÃO / 02

Escolha pelo perfil da carga

Parta do processo, do protocolo e do estado. Compare custo total, manutenção, recuperação e limites do plano para o seu volume.

Carga	Candidatos	Primeiro teste
API HTTP em contêiner	ECS/Fargate, Azure Container Apps, Cloud Run ou VPS/Compose.	Subir imagem, chamar ingress real e verificar conexões ao banco.
Função, webhook ou tarefa curta	AWS Lambda, Azure Functions, Cloud Run functions ou Cloudflare Workers.	Testar timeout, assinatura, entrega duplicada e efeito durável.
Consumidor ou job agendado	Tarefa ECS, Container Apps Jobs, Cloud Run Jobs, fila gerenciada ou serviço no host.	Provar ack, retry, limite de concorrência e destino de falhas.
Edge e origem	Workers ou Cloudflare como DNS, proxy, WAF ou Tunnel diante da origem.	Testar TLS e autorização da borda até o serviço. Medir latência real.
gRPC, WebSocket ou TCP/UDP	Runtime e balanceador compatíveis, Kubernetes ou VM.	Verificar protocolo, idle timeout, drenagem e reconexão na cadeia inteira.
Estado, batch ou GPU	Banco gerenciado, VM, Kubernetes, jobs ou serviço de batch especializado.	Testar volume, checkpoints, capacidade, quota e restauração.

Compare com números do projeto

Declare região, tráfego de pico, armazenamento, tráfego de saída, mínimo de instâncias, rede e retenção de logs. Estime o custo de operar e recuperar. Confirme disponibilidade e preço no provedor antes de contratar ou alterar recursos.

PRIMEIRO ENSAIO

Em homologação, execute uma fatia com um cliente, um armazenamento e uma falha representativa. Se a decisão depende de streaming, fila ou restauração, exercite essa fronteira já no ensaio.

PRODUÇÃO / 03

Prepare acesso sem expor segredos

Você fornece coordenadas, referências e autorização. A pessoa responsável conclui MFA ou consentimento diretamente com o provedor.

Responsável	Entrada e evidência
Você ou o administrador	Repo, revisão, ambiente, conta ou projeto, região, serviço, domínio, limite de custo, janela e ações autorizadas. Forneça o perfil, papel ou runner aprovado.
Bootstrap e CI	Restrinja a confiança OIDC ao emissor, audiência e ref ou ambiente esperado. A identidade de deploy publica e aplica no alvo previsto. Use as referências de segredos do CI ou cofre.
Aplicação e migração	Identidades separadas quando os efeitos diferirem. A aplicação acessa somente as dependências que usa. O job de schema recebe permissão própria e temporária quando viável.
Agente e operador	Confirme principal e alvo no ambiente que executará a mudança e leia um recurso conhecido. Uma consulta de inventário demonstra acesso de leitura.

MCP, CLI e pipeline

Use o MCP oficial para documentação, inventário e triagem quando o catálogo e as permissões cobrirem a tarefa. Valide o publicador, o endpoint, a identidade e as ferramentas. Para deploy reproduzível, prefira o pipeline e o IaC do projeto. A CLI aprovada cobre ações ausentes do conector.

Um login no seu computador pode estar indisponível para o runner, o sandbox ou o processo MCP. Prove o acesso onde ele será usado. Em VPS, confirme a chave pública do host por canal confiável e mantenha a chave privada no agente SSH ou no mecanismo autorizado.

O QUE ENVIAR NO CHAT

Envie nomes de perfis, IDs, referências do cofre, caminho do runner e o escopo autorizado. Insira valores secretos diretamente no cofre ou no mecanismo de acesso aprovado.

Documentação

GitHub Actions / federação de identidade

MCP / práticas de segurança

PRODUÇÃO / 04

Monte um pipeline que prove o deploy

Implemente no CI do projeto. As etapas abaixo ligam a revisão de código à versão realmente servida.

Gate	Execução	Evidência
PR	Testes de comportamento e validação de IaC. Separe código não confiável de credenciais e runners de deploy.	Run e revisão, checks e diff.
Build confiável	Gere o artefato e registre o digest. Promova o mesmo artefato quando o runtime permitir.	Commit, digest ou versão imutável, registry e run.
Infraestrutura	Revise alterações, substituições, acesso, exposição e custo. Aplique no alvo confirmado sob lock ou concorrência controlada.	Plano e IDs dos recursos. Reconciliação se a operação expirar.
Homologação	Aplique a sequência de schema e código. Teste a revisão nova com dados sintéticos e a rota de ingresso real.	401 sem efeitos, replay com mesmo recibo, tenants separados e 409 sem novo job.
Produção	Siga a política de promoção e os limites autorizados. Observe erros, latência e efeitos assíncronos.	Versão, fração de tráfego, janela, métricas, operação e decisão.

CHAT / CONFIGURAÇÃO DO PIPELINE

```
$production-operations
Prepare the staging workflow for the accepted API revision.
Use the protected environment and the existing registry.
Separate untrusted PR checks from federated deployment.
Review triggers, permissions, migrations and recovery.
Validate local changes. Leave workflow activation pending.
```

Se um apply ou deploy expirar, consulte a operação, o estado da infraestrutura e os recursos reais antes de repetir. Confira o digest e o estado servido depois de cada troca de tráfego.

Documentação

GitHub Actions / segurança de workflows

Terraform / planos salvos e revisão

PRODUÇÃO / 05

VPS com Compose ou systemd

Use este caminho para uma API portátil, um runtime específico ou um serviço pequeno com operação explícita do host.

A. Contêiner em VPS

Prepare host suportado, usuário de deploy, firewall, proxy TLS, registry e backups fora da máquina. Em CI, construa a imagem por digest. Confirme host e mounts, valide o Compose sem expor segredos, baixe a imagem e substitua somente o serviço previsto. Verifique digest, API externa, gravação e persistência.

B. Binário ou aplicação supervisionada

Use o gerenciador de serviços existente. Prepare diretório de releases e configuração fora do binário. Valide a unidade, permissões, logs, reinício e troca de versão. Confirme a inicialização após reboot e o procedimento de recuperação via console.

C. VPS atrás do Cloudflare Tunnel

Verifique a saúde da origem e do conector separadamente. Restrinja o acesso direto à origem quando a topologia exigir e teste o domínio pelo caminho público. Faça mudanças de DNS e do serviço em etapas que permitam atribuir falhas.

ABORTAR E RECUPERAR

Pare se a identidade ou a chave do host divergir, se o armazenamento esperado estiver ausente ou se o smoke falhar. Reative a versão anterior compatível e repita a checagem externa e dos dados. Para uma VPS única, registre a janela de indisponibilidade prevista e a medida observada.

Acesso. Host, porta, usuário, fingerprint, rota de bastion ou VPN, registry e console de recuperação. O MCP do fornecedor pode servir ao inventário. Use o SSH ou runner autorizado para operar o sistema convidado.

Documentação

Docker / Compose em produção

OpenSSH / verificação do host

Cloudflare / configuração de Tunnel

DigitalOcean / MCP do provedor

PRODUÇÃO / 06

AWS com ECS, Lambda ou Batch

Use ECS para um serviço em contêiner. Escolha Lambda ou Batch quando o modelo de execução corresponder à carga.

A. API com ECR, ECS/Fargate e banco gerenciado

Confirme conta, região, VPC, target group, serviço e registry. Configure a relação de confiança do CI e separe o papel de execução do ECS do papel da aplicação. Publique o digest, gere a task definition e aplique a estratégia configurada. Prove tarefas prontas, revisão e digest, health do balanceador e o contrato da API.

B. Lambda para webhook ou evento

Publique uma versão imutável e verifique que o consumidor usa o alias quando o rollout depender dele. Comprove assinatura, retry, idempotência e efeito durável. Em filas e eventos, confira versão consumidora, limites, falhas e redrive independentemente do tráfego HTTP.

C. Batch, EC2 e EKS

Use Batch para jobs que precisam de duração, recursos ou paralelismo específicos. Valide fila, imagem, quota, retry e artefato de saída. Em EC2, aplique o roteiro de host. Em EKS, use os checks de cluster da página 31.

ACESSO E ABORTO

Confira conta e ARN efetivos com **aws sts get-caller-identity** no caminho usado pelo deploy. Use um perfil explícito quando ele for o escolhido. Antes de confiar em rollback automático no ECS, confira a estratégia, o detector de falha e a revisão anterior recuperável. Verifique dados e jobs após reverter.

MCP. O AWS MCP Server documenta OAuth e proxy SigV4. Valide escopo IAM, região do endpoint e região do recurso. Use CLI, IaC ou pipeline para a alteração aprovada.

Documentação

AWS / configuração do MCP

ECS / detecção de falha

Lambda / roteamento com alias

PRODUÇÃO / 07

Azure com revisões, slots ou jobs

Escolha a unidade de release conforme o processo e a plataforma que já sustenta a equipe.

A. Azure Container Apps e Jobs

Configure ACR, rede, identidade e acesso a segredos. Em modo de múltiplas revisões, ensaie a revisão ou o label em um caminho de teste e verifique a distribuição efetiva antes de aumentar tráfego. Para schema e batch, registre o resultado da execução do Job e a reconciliação dos dados.

B. App Service com deployment slots

Use slots quando o plano escolhido oferecer o recurso. Valide no slot de staging a configuração aplicável, conectividade, identidade, warm-up e persistência. Antes do swap, confira quais configurações permanecem no slot. Depois do swap, teste o hostname de produção.

C. Functions, AKS ou VMs

Para Functions, valide plano, limite de execução, triggers e armazenamento. Em AKS, confira identidade e namespace antes do rollout. Para um runtime específico em VM, use a trilha de host e a topologia de rede da assinatura.

ACESSO E ABORTO

Informe tenant Entra, assinatura, grupo de recursos, região, registry, IDs de identidade e alvo. Use OIDC no CI quando aplicável. Se o smoke falhar, mantenha ou restaure o roteamento compatível e comprove a revisão servida e os efeitos em dependências.

MCP. Confira o catálogo do Azure MCP Server. A ferramenta documentada para Container Apps lista apps. Use Azure CLI, Bicep/ARM ou o pipeline para executar os passos que o conector disponível não oferece.

Documentação

Container Apps / revisões

App Service / slots

Azure MCP / ferramentas de Container Apps

PRODUÇÃO / 08

Google Cloud com Cloud Run

Use um serviço para a API e uma execução identificável para o backfill. Separe o projeto do workload do projeto que hospeda a federação.

A. Artifact Registry, Cloud Run e Cloud SQL

Confirme projeto e região com a identidade real do pipeline. Acesse a base privada com identidade própria. Em serviço existente, prepare uma revisão com tag e sem tráfego padrão, teste pela rota autenticada, depois ajuste os pesos. Registre digest, revisão, mapeamento e checks na URL principal.

B. Migração em Cloud Run Jobs

Fixe imagem, identidade, base e timeout. Guarde o ID da execução, checkpoints e ledger de migração. Se o CI perder a resposta, consulte a execução e os lotes confirmados antes de iniciar outra. Verifique a compatibilidade com a API e o worker anteriores.

C. Worker pools, GKE e Compute Engine

Considere worker pools para consumidores contínuos quando disponíveis no alvo. Valide escalabilidade e lag. Use GKE ou Compute Engine quando precisar de controle de cluster, protocolo, rede ou host fora do contrato do runtime gerenciado.

UM PROJETO ERRADO BLOQUEIA O DEPLOY

A federação pode morar em outro projeto. Confirme o projeto do recurso nos comandos e a identidade no runner. Se o runner apontar para produção em uma tarefa de homologação, mantenha o tráfego e corrija a configuração antes de escrever.

MCP. Escolha o servidor oficial do produto e confira endpoint, IAM e OAuth. O catálogo inclui Cloud Run e GKE. Use gcloud, CI ou IaC para as etapas autorizadas de release.

Documentação

Google Cloud / federação no CI

Cloud Run / revisão e tráfego

Cloud Run / executar jobs

Google Cloud / catálogo MCP

PRODUÇÃO / 09

Cloudflare com Workers e origem

Use Workers para cargas compatíveis com o runtime de borda. Se a aplicação exige outro runtime ou protocolo, avalie Containers ou um backend externo.

A. Workers com D1, Queues e bindings

Fixe Wrangler e o ambiente. Confira IDs de banco, fila e namespaces em cada revisão. Valide o bundle localmente e teste com recursos isolados. Para código que permite versão gradual, faça o upload, verifique a versão e observe resultados de API, D1 e fila separadamente. O peso HTTP não comprova o comportamento do consumidor.

B. Durable Objects e migração de classe

Aplique mudanças de ciclo de vida de classe pelo fluxo direto documentado. Um rollback de Worker não atravessa esse tipo de migração. Separe a mudança de código compatível quando precisar de rollout gradual e prepare reparo ou recuperação do estado para a mudança de classe.

C. Containers e origem híbrida

Verifique limites, persistência e instâncias efetivamente ativas no caminho de Containers. Em proxy ou Tunnel para uma origem externa, prove o deploy da origem e a rota de borda em separado. Teste domínio, certificado, autorização e comportamento.

ACESSO E ABORTO

Informe conta, zona, Worker, ambiente, domínios e IDs dos bindings. Injete o token restrito no CI. Se uma versão de homologação apontar para D1 de produção, bloqueie testes e rollout com esses bindings. Confirme também o alvo da revisão de recuperação.

MCP. O servidor de documentação serve à consulta. O servidor da API cobre operações segundo o escopo concedido. Prefira Wrangler e pipeline para publicação reproduzível.

Documentação

Cloudflare / servidores MCP

Workers / rollbacks e bindings

Durable Objects / ciclo de vida

Containers / rollouts

PRODUÇÃO / 10

Kubernetes, VMs e serviços com estado

Quando o workload precisa de controle de protocolo, rede, armazenamento ou agendamento, valide o controlador e o caminho real do cliente.

Cenário	Implantação e prova
EKS, AKS ou GKE	Confirme contexto e namespace. Revise manifests renderizados, probes, quotas, rede e identidade. Use Deployment para a API, Job para a tarefa finita e StatefulSet onde identidade e armazenamento estáveis forem necessários.
GitOps	Revise a mudança no repositório que governa o cluster. Verifique reconciliação e digest nos Pods prontos. Corrija a configuração desejada quando reverter para evitar que o controlador republique a versão problemática.
Frota de VMs	Publique imagem ou pacote versionado, prepare capacidade, drene instâncias e valide health, rotas e reinício. Conserve o artefato anterior e o console de recuperação.
Conexões longas	Exercite gRPC, WebSocket ou TCP/UDP pelo balanceador real. Verifique negociação, timeout ocioso, drenagem e reconexão com versões coexistindo.
Banco ou broker	Use credenciais, volumes e políticas de replicação explícitas. Restaure em destino isolado e valide dados, dependências, chaves e tempo de recuperação.

Cutover de DNS e TLS

Prepare os dois destinos, certificados e identidade do host. Observe respostas de DNS e conexões durante a transição. Confirme que as duas origens podem coexistir enquanto houver clientes com respostas antigas. Preserve capacidade e recuperação durante o período previsto.

PROVA DO SERVIÇO

Um Pod pronto, um target saudável ou um DNS atualizado estabelece apenas aquela condição. Exercite o contrato da API por sua rota real e confira a versão e o efeito no estado.

Documentação

Kubernetes / Deployments

Kubernetes / probes

PRODUÇÃO / 11

Migre os dados com recuperação

Na API de recibos, a regra muda o escopo de uma chave existente. O plano precisa cobrir escritores antigos e os efeitos já confirmados.

Fase	Exemplo para tenant-receipts
Expandir	Crie a estrutura compatível e exercite gravações da revisão anterior. Estime duração, locks, concorrência e espaço. Use base de ensaio representativa.
Preencher	Associe registros ao tenant por uma fonte verificada. Use lotes, checkpoints e uma execução coordenada. Separe casos ambíguos e confira chaves, duplicatas e totais.
Promover	Atualize os escritores, trate alterações concorrentes e valide a API com as duas revisões conforme o contrato. Faça a troca de tráfego quando os critérios forem atendidos.
Consolidar	Aplique restrições definitivas após confirmar os escritores ativos e a janela de recuperação. Trate remoção irreversível como uma etapa com verificação e escopo próprios.

Escolha a recuperação antes da promoção

Se o código anterior ainda escreve e lê a estrutura atual, restaure a rota compatível. Se houver incompatibilidade ou corrupção, avalie reparo adiante ou restore para um destino isolado com reconciliação de novas escritas.

BACKUP E RESTAURAÇÃO

Defina a perda de dados aceitável (RPO) e o tempo de recuperação (RTO). Ensaie restauração, acesso, consistência e fluxo crítico. Registre a perda e o tempo medidos com o escopo e o ponto recuperado.

Em consumidores, confira quais mensagens foram confirmadas, repetidas, estacionadas ou ainda estão em execução. Replay exige um limite e uma checagem de idempotência.

Documentação

AWS Well-Architected / recuperação

Google SRE / releases canário

PRODUÇÃO / 12

Opere, contenha e feche o ciclo

Defina os sinais, as janelas e quem pode agir. Os limites abaixo são campos do projeto e variam com a carga.

Momento	Ação e registro
Antes	Confirme versão atual, linha de base, checks críticos e recuperação. Defina janela e evidência mínima. Em baixo tráfego, prepare uma transação sintética.
Durante	Compare erro, latência, saturação, conexões, lag e efeitos persistidos por versão. Registre contagens, janela e condições que interrompem a promoção.
Incidente	Estabeleça impacto e linha do tempo UTC. Contenha dentro da autoridade, por exemplo pausando o rollout ou o consumidor. Verifique recuperação pela API, fila e dados.
Depois	Ligue commit, digest, run, ambiente, revisão observada, tráfego e checks a PROOF. Atualize PRODUCTION com o runbook e PROCESS com responsável e pendências.

CHAT / RELEASE AUTORIZADO

```
$production-operations
Deploy the accepted image digest to the named staging service.
Use the approved pipeline and the scoped deploy identity.
Verify the serving revision and the tenant-receipts scenarios.
Hold promotion if verification fails or is inconclusive.
Record actual effects, limits and the next action.
```

Use o modelo **deployment-brief.md** para a passagem de contexto e **operations-record.md** para evidência e runbook. Um incidente pode gerar uma investigação com Diagnose Defect e uma nova change quando o contrato precisar mudar.

Documentação

Google SRE / gestão de incidentes

Teknolista / coleção e guia de operação

20 / EDIÇÃO PONTUAL

Use um prompt curto com critério

Corrigir “Salvar notaa” para “Salvar nota” em um botão conhecido é um caso de baixo impacto quando o significado e a ação permanecem iguais.

CHAT / EDIÇÃO DIRETA COM UI DESIGN QUANDO ÚTIL

```
$ui-design
In the existing note editor, fix only "Salvar notaa"
to "Salvar nota" in the Portuguese string resource.
Keep the action, layout, identifier and other locales.
Inspect the rendered label if the environment allows.
Show the diff and checks you actually ran.
```

01 **Confira o contrato.** Identifique o recurso de texto certo, seu uso e o termo estabelecido. Uma skill é opcional se a edição for inequívoca e não exigir orientação especializada.

02 **Faça a mudança focada.** Troque o texto no local oficial. Se houver teste de texto ou catálogo de traduções que dependa dele, avalie o ajuste correspondente. Preserve IDs usados por automação.

03 **Confirme o efeito.** Leia o diff e valide a compilação ou renderização relevante. Se não houver ferramenta de UI, relate o que foi verificado e deixe a inspeção visual pendente.

QUANDO MUDAR DE MODO

Se a correção muda consentimento, aviso legal, nome acessível com significado distinto ou a operação do botão, estabeleça o contrato do comportamento antes de editar. “Mudar só uma string” pode ser uma alteração importante.

Para um ajuste desse tamanho, o próprio pedido e o contrato existente podem ser suficientes. Se o projeto exige OpenSpec para esse tipo de mudança, siga a política.

21 / VIBE CODING E PROMOÇÃO

Explore com dados sintéticos

Compare duas maneiras de mostrar o estado vazio do app. O resultado da exploração é uma decisão de design sustentada por observação.

CHAT / PROTÓTIPO ISOLADO

```
$ui-design
Prototype two empty-state layouts for the notes list.
Use a disposable local preview with synthetic notes.
Keep product files and services outside the experiment.
Compare first-action clarity and keyboard navigation.
Stop after two variants and record what you observed.
```

Use `$technical-research` se a dúvida for técnica. Exemplo: verificar se a biblioteca de armazenamento permite reabrir o banco em modo offline depois de um erro. Defina hipótese, observação e condição de parada. Use `$openspec-explore` se a discussão já pertence a uma possível change.

Depois do protótipo, escolha o que manter

01 **Registre o aprendizado.** A opção com uma única ação principal facilitou o primeiro passo no cenário observado. Vincule o resultado, condições e limites à decisão do produto.

02 **Crie o contrato se houver entrega mantida.** Se o escolhido virar produto, proponha a change. Especifique regras de estado, textos, foco, casos de erro e critérios de aceitação no dono de cada artefato.

03 **Reaproveite com critério.** Decida o que descartar, reescrever ou portar. Revise dependências, dados temporários e atalhos. Verifique o comportamento na implementação real.

EXEMPLO DE MUDANÇA DE RISCO

Se o protótipo começar a usar notas reais, credenciais ou serviços externos, o trabalho ganha outras fronteiras. Confirme autoridade, dados e verificações antes de ampliar o experimento.

22 / TRILHAS

Adapte ao tipo de software

O ciclo de intenção, contrato e verificação serve a diferentes domínios. Selecione os artefatos e as skills conforme as fronteiras reais.

Superfície	O que especificar	O que observar
Web e front-end	Fluxo, estados, teclado, semântica, responsividade e localização. UI Design para tela e interação.	Navegação e erro no navegador real. Testes de comportamento no runner existente.
Android nativo	Ciclo de vida, estado persistido, permissões, APIs e variantes. Use os helpers da página 101 conforme a stack.	Fronteiras de armazenamento e jobs, emulador ou dispositivo e versão de API.
HTTP e serviços	Consumidores, compatibilidade, identidade, erros, retries, migração. HTTP API e Cybersecurity conforme tarefa.	Controle permitido e negado, efeitos duráveis, integração e cliente real quando exigido.
MCP	Versão do protocolo, SDK, transporte, capacidades, identidade, schema, limites e efeitos. MCP Server para a implementação.	Cliente e transporte reais quando possível. Repetição após falha, efeitos negados e versão suportada.
CLI, biblioteca ou dados	Entradas, saídas, códigos de retorno, erros, estabilidade, esquema, reversão e volumes.	Chamador real ou fixture representativa. Compatibilidade, repetição, recuperação e estados intermediários.
Desktop, embarcado e outros	Acesso, runtime, falhas, plataforma, recursos e condições de operação. Use as skills genéricas e fontes do framework.	Evidência na plataforma alvo. Registre o limite de um simulador ou build quando faltar execução real.

PROJETOS COM CÓDIGO GERADO

Se um schema ou DSL governa o resultado, edite essa fonte pelo fluxo oficial e regenere. Inspecione o diff e valide consumidores. Conserve intenção e decisões onde o schema não tem como expressá-las.

23 / CONTINUIDADE

Retome, refine e resolva desvios

Uma nova sessão deve conseguir retomar pelo repositório e pelo registro de tarefa, com acesso ao escopo autorizado.

CHAT / RETOMADA DE UMA TAREFA

```
$software-delivery
Resume the accepted change from docs/process.md.
Confirm repository, revision, change ID and next task.
Read its governing contract and the actual check evidence.
Continue the next authorized task and report pending gates.
```

Se acontecer	Faça
Skill ausente ou duplicada	Confira raiz, caminho, name, versão e recursos. Atualize ou reabra a sessão conforme o host. Para verify, confira perfil expandido e regenere a integração com autorização.
Status completo, teste ausente	Trate status como progresso dos artefatos. Mantenha a verificação pendente com ambiente, dono e próximo passo.
Regra mudou no meio da tarefa	Use $openspec-update-change para reconciliar o delta, design e tarefas. Revalide, reavalie a prontidão e repita checks afetados.
Duas changes na mesma capacidade	Compare deltas e dependências, estabeleça dono e ordem de integração. Reveja o segundo delta contra a spec atualizada antes de arquivar.
Um incidente revelou lacuna	Reproduza com Diagnose Defect. Atualize o contrato se a regra mudou, ou corrija a implementação se o contrato continua válido. Adicione verificação que observe a falha.

Se a equipe deseja agentes em paralelo, divida fronteiras com contratos estáveis, propriedade de arquivos e uma ordem de integração. Valide o resultado conjunto. Para melhorar as instruções, use Agent Instructions e compare tarefas semelhantes por qualidade, rework, tempo e efeitos indesejados.

PARTE II / PLANEJAMENTO

Da ideia ao primeiro alvo real

Use as páginas 3 a 37 como procedimento comum. Os laboratórios seguintes acrescentam contratos, comandos e provas específicos da plataforma. Se estiver começando, execute uma jornada mínima completa antes de ampliar o escopo.

Escolha seu ponto de entrada

- **Produto novo.** Abra a raiz do futuro produto e rode um smoke do esqueleto. Use $penta-pro para confirmar a jornada e os limites, registre o delta no OpenSpec e use $software-delivery na primeira fatia.
- **Produto existente.** Abra o snapshot aceito, reproduza a jornada atual e leia os contratos de clientes e dados. Para uma mudança material, preserve cenários vigentes e escreva somente a diferença. O laboratório de recibos, na página 20, mostra essa evolução.
- **Hipótese de interface.** Use o experimento da página 35 e $ui-design para web ou Android. Dê ao agente dados sintéticos, uma pergunta e um destino descartável. Se a hipótese funcionar, declare quais comportamentos terão manutenção e comprove-os na entrada real antes de integrar.
- **Incidente.** Use $diagnose-defect para reproduzir com um controle funcional e localizar a primeira fronteira que falha. Em serviço hospedado, $production-operations verifica a revisão que recebe tráfego e ajuda a conter e recuperar o ambiente autorizado.

Escolha a primeira prova

- **Android e desktop.** Instale o pacote no alvo escolhido, abra e grave algo que ainda exista depois de reiniciar. Mantenha uma instalação anterior com dados quando o produto já estiver distribuído.
- **Web.** Abra a URL de preview em navegador real e atravesse a fronteira entre interface, identidade e dados. Teste teclado, foco, erro e retorno à jornada.
- **HTTP e MCP.** Use o cliente real ou de protocolo, uma identidade permitida e uma negada. Confira resposta e estado persistido de cada tentativa.
- **Fila, CLI ou firmware.** Use a entrada real da ferramenta, declare códigos, efeitos e limites. Exercite interrupção, reexecução e a plataforma de distribuição prevista.

Como ler os exemplos

Os blocos de chat e artefatos usam inglês, como os pacotes da coleção. Comandos de shell usam a raiz indicada. Substitua caminhos, versões da toolchain, identidades, endpoints e tarefas por valores do seu projeto. Fixe-os no piloto. As matrizes indicam provas a executar.

Pratique o fluxo completo com a CLI de recibos

CASTAS / PREPARAÇÃO

Prove o ambiente antes da feature

A primeira automação do Castas chamou sdkmanager antes de preparar o SDK. Em outro momento, o sandbox ocultou /dev/kvm, embora o host tivesse aceleração. Um preflight por contexto teria revelado os dois limites antes da implementação.

Monte uma sequência curta por alvo

- **Clone limpo.** Confira raiz, HEAD, estado do Git e versões do instalador, OpenSpec, skills e ferramentas. Recrie a instalação e confira a descoberta pelo host. Registre o arquivo que governa a próxima tarefa e peça uma leitura com $penta-pro.
- **Ambiente.** Execute o mesmo wrapper e SDK usados no trabalho real. Confira recursos, cache, espaço, permissões, portas e serviços. Faça a checagem no shell do agente, no host e no runner que realmente executará o passo quando forem diferentes.
- **Teste mínimo.** Compile e instale o app ou inicie o serviço. Rode uma asserção da jornada no alvo e registre revisão, variante, comando, alvo e saída. Configure timeout e compare versões observadas com as esperadas.

Android em Linux, no contexto que inicia o AVD

```
"$ANDROID_HOME/emulator/emulator" -accel-check
# Aponte para o volume gravável usado pelo AVD.
df -h "$ANDROID_AVD_HOME"
adb devices -l
adb -s "$TEST_SERIAL" shell getprop sys.boot_completed
adb -s "$TEST_SERIAL" shell getprop ro.build.version.sdk
adb -s "$TEST_SERIAL" shell getprop ro.product.cpu.abi
```

Defina ANDROID_AVD_HOME e TEST_SERIAL antes dos comandos. Avance com o serial previsto em estado device, boot igual a 1 e API/ABI esperadas. Em aparelho físico, use as checagens do ADB e aponte para um aparelho de teste autorizado.

Se a primeira prova falhar

Isole host, sandbox, disco, imagem, SDK, ADB e app. KVM acelera a CPU virtual e pode coexistir com renderização SwiftShader. Boot, ANR, instrumentação e captura visual têm resultados próprios. Separe a captura opcional da suíte. Em scripts, use Bash quando depender dele e evite nomes reservados do shell.

Auditoria retrospectiva do Castas, pp. 9–12 e 23.

CASTAS / CONTRATO E PROVA

Porte regras com vetores verificáveis

No Castas, uma referência de interface divergia do comportamento executável. A confirmação do domínio exigiu comparar as fontes e registrar o que o aplicativo Android deveria preservar.

Preencha uma ficha para cada divergência

```
Behavior: compute a dimension score
Reference: web source at <revision and path>
Conflict: wireframe and executable behavior disagree
Decision: use weights 3, 2, 1, 0 after owner acceptance
Check: compare all 25 possible totals per dimension
Proof: <command, two revisions, fixture, result, limits>
```

Faça o mesmo para traduções, assets, ordenação, permissões e estado. Na auditoria, a paridade histórica cobriu 120 traduções, 25 totais por dimensão, 101 contagens de emoji e três assets. Em outro produto, selecione os vetores pela regra acordada e registre quais diferenças de plataforma foram aceitas.

Faça a asserção derivar do contrato

- **Estado e progresso.** O primeiro teste Compose aceitava o contador errado. No Castas, a pergunta 9 deve mostrar a primeira questão da dimensão seguinte e progresso total 9. Injete a regressão numa cópia isolada e confirme que a asserção falha.
- **Persistência.** Confirme sucesso após commit. Provoque falha e confira que o último resultado íntegro ainda aparece. Teste rotação e morte do processo como eventos distintos.
- **Apresentação.** Exercite texto ampliado, contraste em cada estado, locale, orientação, janela pequena e tablet. Use semântica automatizada e faça uma jornada com a tecnologia assistiva real.
- **Aplicação da skill.** Registre versão e escopo instalado, seleção no host e saída observável. Um teste que reabre o DataStore evidencia o exercício da fronteira. A descoberta da skill evidencia disponibilidade.

Repita em outros produtos

Em uma API portada, compare consumidores e efeitos com fixtures antigas. Em desktop, preserve o formato e teste o pacote instalado. Em web, verifique o navegador e a origem de armazenamento. Em MCP, compare descobertas, esquemas e chamadas nos clientes suportados.

Auditoria retrospectiva do Castas, pp. 6–8, 11, 16, 18 e 20.

CASTAS / ENTREGA

Feche o caminho até o usuário

O relatório registrou build, instrumentação, verificações de splits e CI em revisões específicas. Os controles abaixo transformam as lacunas encontradas em trabalho com responsável e critério de aceite.

- **Mantenedor e documentação.** Valide links e comandos em clone limpo. Compare README, privacidade, manifesto efetivo e ficha da loja. No Castas, uma frase importada sobre localStorage contradizia DataStore e um link de licença estava quebrado. Atualize resultados antigos com data e revisão, preservando seu contexto histórico.
- **CI e Git.** Use branch curta e PR conforme política. Prepare toolchain antes dos checks, valide o wrapper e identifique o artefato produzido. Separe compilação da suíte e execução em dispositivo. Confira acesso ao workspace, ao índice Git e ao remoto. Use git check-ignore para paths esperados e versione configuração, specs e mudanças do OpenSpec.
- **Privacidade e recuperação.** Rastreie dado em memória, banco, backup, migração, logs, exportação e clipboard. Um aviso temporário de cópia deixa o dado no clipboard. Exercite exclusão pelo app, inclusive falha e reabertura, quando fizer parte do produto. Use entradas fictícias em evidências compartilhadas.
- **Distribuição.** Confirme direitos de assets e avisos. Registre hash, variante, identidade, versionCode, chave e trilha. Exercite upgrade no canal real em aparelho físico. Para 16 KiB, quando aplicável, confronte alinhamento do APK, segmentos ELF das bibliotecas nativas e execução no ambiente compatível. Defina stop de rollout, suporte e hotfix preservando dados.

Do diagnóstico à operação

Registre o que já foi observado, o que depende de outro alvo e quem fecha cada item. Um verde do CI prova seu job e sua revisão. A execução local prova seu dispositivo e pacote. Para decidir a promoção, o responsável precisa da instalação e da jornada observadas pelo canal escolhido.

Use os registros de PROCESS para a próxima tarefa, PROOF para a evidência e PRODUCTION para o caminho de distribuição e recuperação. Regras automatizáveis entram no CI. Rotinas reaproveitáveis entram na skill responsável quando o piloto indicar benefício.

Auditoria retrospectiva do Castas, pp. 12–25.

Android Developers · compatibilidade com páginas de 16 KiB

CASTAS / EVIDÊNCIA POR REVISÃO

Leia a evolução da evidência

A auditoria de 9 de novembro de 2026 separa o primeiro APK, os reparos, o dispositivo virtual e o CI. Os resultados desta página descrevem aquele projeto e aquelas revisões.

Da primeira entrega à candidata

- **Primeira entrega, 2a133da.** O registro histórico mostra seis testes JVM e compilação do teste Compose. O primeiro AVD ficou bloqueado. A execução no Android era uma pendência com ambiente e próximo passo próprios.
- **Fechamento local, 14ab318.** A revisão corrigiu persistência, fluxo e compatibilidade. O XML retido confirmou nove testes JVM. O relatório registrou cinco testes instrumentados em cada um dos alvos API 26 e API 36 e a checagem do bundle e de APKs derivados. Cada registro tem seu alvo e alcance.
- **CI, 9ce150a.** A preparação do SDK corrigiu a falha inicial de sdkmanager. O run 42590362068 concluiu build, JVM, lint e AAB. A execução dos testes instrumentados tinha evidência local separada.

Ligue o achado ao controle certo

O contador errado passou por um teste que copiava a implementação. Compare a regra aceita, a revisão defeituosa e a corrigida. Use o caminho de paridade para cálculo e catálogos e uma jornada de dispositivo para estado, restauração e acessibilidade.

Ao fechar um release, confirme quem produz a evidência de aparelho físico, TalkBack, upgrade pela Play e assinatura. O relatório identificava esses pontos como pendências próprias. Associe cada evidência ao pacote que os testadores realmente instalaram.

Use no próximo projeto

Em PROOF, mantenha uma linha por revisão e fronteira, com cenário, comando, artefato, ambiente, resultado e dono da próxima ação. A mudança de revisão reabre as checagens que o diff pode afetar. Preserve o resultado anterior com seu escopo.

Auditoria retrospectiva do Castas, pp. 5, 7–13, 16–17 e 20–25.

Procedimento de paridade e dados de teste

Checklist de entrega por tipo de alvo

ENGENHARIA / CONTROLES PROPORCIONAIS

Projete falhas que o teste consiga revelar

Use $penta-pro para decisões materiais, $behavior-testing para verificações de comportamento fora de Android nativo e $cybersec quando a mudança altera uma fronteira de confiança. Comece pela consequência que um teste raso deixaria passar.

Contrato, código e dados

- Declare invariantes observáveis, como um único efeito por operação, isolamento entre tenants e preservação de dados em upgrade. Localize quem controla cada efeito. Compare resposta, autorização e estado persistido.
- Valide entradas e saídas no limite responsável. Planeje timeout, cancelamento, repetição e resultados ambíguos. Quando houver escrita com retry, ligue a identidade da operação ao efeito durável e prove reexecução após uma resposta perdida.
- Exercite entrada válida, negativa relevante, compatibilidade e recuperação. Derive o oráculo do contrato. Teste a revisão anterior defeituosa ou injete uma falha controlada quando isso mostrar que a asserção percebe a regressão.

Cadeia de entrega

- Use toolchain e dependências registradas, lockfiles, fontes verificadas e identidades de CI com menor privilégio. Separe o job que avalia código de PR do job autorizado a acessar segredos e publicar. Revise workflows, permissões, artefato e destinos.
- Faça o teste no limite que sustenta a decisão. Unitário protege uma regra isolada. Integração usa o componente que pode falhar. Uma jornada de release testa o pacote ou a revisão efetivamente servida, com dados e identidades descartáveis.
- Inclua estratégia de atualização e recuperação para estado durável. Aplique rollback de código quando a revisão anterior consegue ler e escrever no estado atual. Para mudanças incompatíveis, ensaie uma correção adiante ou restauração autorizada.

Feche a evidência

No PR ou PROOF, relacione cenário, revisão, artefato, comando, alvo, resultado e pendências. Em riscos materiais, uma revisão em contexto separado pode testar o contrato, a eficácia dos checks e o risco residual. Use o gate real do projeto para integração.

NIST · práticas de desenvolvimento seguro

GitHub Actions · uso seguro

GIT / ENTREGA DA REVISÃO CERTA

Passe a candidata para revisão

Mantenha proposta, diff e prova ligados à revisão que vai integrar. Use $peer-review para padrões e comportamento, $cybersec para riscos materiais e $git-delivery para o commit, push ou PR autorizado.

Antes de publicar

- Confira branch, base, staged/unstaged, arquivos novos e commits de saída. Preserve seleções e alterações de outras pessoas. Revise hooks e workflows que um push pode acionar.
- No PR, inclua objetivo, change, comportamento anterior e candidato, checks executados na revisão certa, risco residual e recuperação. Para mudanças de dados ou API, vincule a compatibilidade e os checks com a versão antiga.
- Verifique o commit no destino e a execução do CI naquele hash. A integração segue os responsáveis e checks do projeto. Release e promoção têm evidência própria.

Quiz de compreensão quando solicitado

$pr-quiz prepara perguntas sobre decisões da revisão. Para publicar, capture o head antes de ler o diff e passe o repositório e o SHA revisado. Use o caminho real da skill instalada e um JSON validado.

```
gh pr view 123 --repo OWNER/REPO --json url,headRefOid
QUIZ_PY=.agents/skills/pr-quiz/scripts/publish_quiz.py
python3 "$QUIZ_PY" 123 /tmp/quiz.json --dry-run
python3 "$QUIZ_PY" 123 /tmp/quiz.json \
  --repo OWNER/REPO --head "$REVIEWED_SHA"
```

Defina REVIEWED_SHA com o SHA completo que embasou as perguntas. A versão 1.3.0 fixa a URL do PR e confere head e corpo antes da escrita. Confirma a edição depois. Se houver divergência após envio, inspecione o PR antes de repetir. A conferência por leitura tem uma janela de concorrência.

O quiz ajuda a conversar sobre decisões. A revisão de código, a avaliação de segurança e a decisão de merge seguem seus procedimentos próprios.

GitHub CLI · seleção do PR e campos JSON

ANDROID / CONEXÃO COM O FLUXO

Configure um projeto Android próprio

A trilha usa Kotlin, Compose e Room. No exercício geral, notes-create-edit ensina a escrever uma mudança que inclui edição. Aqui, add-offline-notes cria e lê notas primeiro, e a segunda change adiciona a fixação com migração. Use um repositório próprio para cada laboratório.

Siga a preparação compartilhada

- Execute o passo a passo do OpenSpec na página 5 e instale as skills conforme a página 6, na raiz do app. O guia fixa OpenSpec 1.17.0 para os comandos e deltas atuais. Os registros históricos do Android citam 1.16.0. Confira o schema, os comandos e os nomes gerados na edição que você instalar.
- Para repetir o setup, registre a revisão com git -C "$SKILLS_REPO" rev-parse HEAD e fixe a edição do instalador. Preserve SKILL.md, referências, modelos, licença e scripts. Se a sessão já estava aberta, confirme a descoberta na próxima sessão.
- Faça uma alteração direta de texto como controle do piloto. Explore a posição do botão Nova nota com um protótipo separado. Use SDD nas duas mudanças deste capítulo, pois elas introduzem persistência e depois alteram uma base existente.

Preencha os cinco pilares com o briefing Android

```
$penta-pro
Create the five pillar docs for Notas de Bolso.
Use the agreed offline journey and exclusions from the brief.
Link the OpenSpec capacity, scenarios, and tasks.
Mark unresolved decisions and planned evidence.
Review the saved draft. Specification only.
```

Use a divisão de responsabilidades da página 4. Localize a regra de entrada em branco no delta, a política de backup na decisão técnica aplicável, a prova de reabertura em PROOF e a autoridade de execução em PROCESS. A partir desse mapa, siga a base Android, o briefing, a UI e os arquivos da change.

Quando a stack mudar

Acione $android-module-structure para build e módulos e $android-data-layer para o repositório. Use $android-error-handling para falha e cancelamento entre camadas. Adicione $android-di-koin ou $android-background quando a stack exigir injeção ou trabalho durável. O catálogo traz os três donos dos testes.

01 / ROTEIRO

Seu primeiro app em SDD

Use o laboratório como referência. No seu projeto, troque a jornada, as regras e as evidências pelo que seu público realmente precisa.

SDD significa desenvolvimento guiado por especificações. Você combina o comportamento esperado e a forma de comprová-lo antes de mudar o código. O OpenSpec organiza as mudanças. As skills dão instruções ao agente para tarefas específicas.

Preparar ambiente > **Acordar uma change** > **Implementar e testar** > **Revisar e integrar** > **Distribuir e aprender**

Marco	Você pode avançar quando
Ambiente	O projeto inicial compila e abre no emulador. As versões e os comandos estão registrados.
Contrato	Uma pessoa consegue descrever o fluxo, os limites e os casos de falha a partir da proposta e do delta.
Entrega local	A nota salva sem rede reaparece após fechar e reabrir o app. Falhas e entradas inválidas têm cobertura.
Entrega para usuários	O artefato da revisão esperada instala na trilha escolhida. O upgrade preserva notas existentes.
Aprendizado	Há um registro de custo, tempo, retrabalho, falhas e decisão sobre o que mudar no processo.

PARA QUEM ESTÁ COMEÇANDO

Faça as etapas na ordem até a revisão local. Use o capítulo de Google Play quando quiser distribuir. Os exemplos de resultado descrevem o que você precisa observar durante seu piloto.

02 / AMBIENTE

Instale a base de trabalho

O roteiro usa um computador de desenvolvimento, um emulador e, na etapa de distribuição, ao menos um aparelho físico.

01 **Android Studio e SDK.** Instale a distribuição oficial. No SDK Manager, obtenha a plataforma escolhida para compileSdk, Android SDK Platform-Tools e a imagem de sistema para os testes. O assistente do Studio informa componentes ausentes.

02 **JDK e Gradle.** Use o JBR do Android Studio para o IDE. Confira o JDK escolhido para rodar o Gradle e a compatibilidade com AGP e wrapper. Alinhe o JDK do terminal e da CI ao usado no Studio.

03 **Emulador.** No Device Manager, crie um Android Virtual Device (AVD) com imagem compatível com o computador. Inicie e desbloqueie o aparelho. Configure um AVD no minSdk escolhido, quando houver imagem disponível, e outro recente.

04 **Git, Codex e Node.js.** Configure sua identidade Git e abra o projeto como workspace no Codex. Para a instalação da coleção com Skills CLI, siga o requisito de Node.js do README. O roteiro adota Node.js 22.20 ou posterior em uma linha suportada.

TERMINAL / PRÉ-REQUISITOS

```
git --version
java -version
node --version
npm --version
adb devices -l
```

Se adb estiver ausente, confira o diretório **platform-tools** do SDK e o PATH do terminal. Se o dispositivo aparecer como **unauthorized**, confirme a conexão no aparelho de teste.

Android Studio e SDK

JDK do Gradle e do Android Studio

Criação de dispositivos virtuais

02 / AMBIENTE

Crie e execute o projeto inicial

Comece por uma base executável. Isso permite distinguir uma falha da instalação de uma falha da implementação.

01 **Crie o projeto.** No Android Studio, escolha New Project e um template Compose de atividade vazia. Use Kotlin e Gradle Kotlin DSL. Para o laboratório, escolha com.example.notasdebolso e minSdk 26. Para um produto real, decida o ID definitivo e as versões com base no público e na distribuição.

02 **Espere a sincronização.** Abra o diretório que contém settings.gradle.kts e gradlew. Deixe o Studio resolver as dependências do template e rode a configuração app no emulador. Confirme que a tela inicial aparece.

03 **Reproduza pelo terminal.** Na raiz do app, use o Gradle Wrapper versionado. No Windows, substitua ./gradlew por .\gradlew.bat. Ajuste os comandos ao nome do módulo e da variante do projeto.

TERMINAL / LINHA DE BASE PARA O MÓDULO APP

```
./gradlew --version
./gradlew :app:tasks --all
./gradlew :app:assembleDebug :app:testDebugUnitTest :app:lintDebug
adb devices -l
```

Observe os relatórios. Uma tarefa de teste com **NO-SOURCE** indica ausência de testes encontrados naquele escopo. Registre esse estado separadamente de um teste executado com sucesso.

CHECKPOINT

Registre Gradle, AGP, Kotlin, JDK, minSdk, compileSdk, targetSdk, API e ABI do emulador, além dos resultados. Confira o .gitignore. Versione o wrapper, catálogos e fontes. Mantenha local.properties, chaves e segredos fora do Git.

03 / CONTEXTO DO PROJETO

Escreva um AGENTS.md curto

Adapte AGENTS-template.md da coleção. Coloque as regras operacionais na raiz do app e registre as decisões onde elas são mantidas.

AGENTS.MD / RECORTE PARA O LABORATÓRIO

```
# Notas de Bolso
- Use English for specs and engineering records.
- Read docs/process.md and the accepted OpenSpec change.
- Use OpenSpec for material behavior and data changes.
- Report the scenario, test command, revision, and outcome.
- Test locally with synthetic notes and a disposable device.
- Preserve persisted notes on write failure and on upgrade.
- Use the real Gradle tasks recorded in docs/process.md.
- Git publication and Play promotion follow team authority.
```

Adicione donos, versões, caminhos, dispositivo de teste e checks obrigatórios. Revise regras por diretório quando existirem. O agente pode escolher detalhes técnicos reversíveis dentro do escopo acordado.

OPENSPEC/CONFIG.YAML / INTEGRE À CONFIGURAÇÃO EXISTENTE

```
schema: spec-driven
context: |
  Read AGENTS.md. Use English for engineering artifacts.
rules:
  specs:
    - Specify observable behavior and failure cases.
  tasks:
    - Map work to scenario and executable evidence.
```

O QUE VERSIONAR

No arranjo local deste guia, mantenha config, specs, changes e histórico de OpenSpec no Git. Se as skills geradas forem ignoradas, documente versão, perfil e comando de instalação. Exclua credenciais, APKs temporários e logs com dados pessoais.

Template de instruções da Teknolista

04 / PRODUTO

Defina a jornada de Notas de Bolso

O app didático captura uma nota sem conta ou rede. A primeira entrega cobre uma jornada pequena, observável e reversível.

Decisão	Acordo do laboratório
Usuário e resultado	Uma pessoa quer anotar um texto e ler a nota mais tarde, mesmo sem rede.
Caminho principal	Lista vazia → Nova nota → Digitar → Salvar → Lista → Fechar e abrir → Ler a nota.
Entrada e ordem	Texto com algum caractere diferente de espaço em branco. Preserve o texto válido como foi digitado. Notas mais novas primeiro, com desempate estável.
Erro e repetição	Se a gravação falhar, mantenha o rascunho no editor e deixe tentar de novo. Ignore acionamentos adicionais enquanto uma gravação estiver em andamento.
Fora da primeira entrega	Editar, excluir, fixar, sincronizar, logar, exportar, compartilhar e restaurar rascunho após morte do processo.
Privacidade	Persistência privada do app, com política explícita de exclusão da base de notas de backup e transferência no escopo suportado. Sem SDK de analytics no laboratório.

01 **Troque o tema, mantenha o método.** Em seu app real, identifique público, hipótese de valor e uma primeira jornada que atravesse UI e estado persistente.

02 **Resolva decisões que afetam o comportamento.** Confirme o que significa “salvo”, como tratar uma falha e como testar que os dados continuaram íntegros.

PONTO DE DECISÃO

Se a sua primeira jornada exigir conta, pagamentos, informação sensível ou integração de backend, aumente a profundidade da especificação e dos testes antes de trabalhar nessa fronteira.

04 / PRODUTO

Especifique as telas e os estados

O layout pode evoluir durante o piloto. Defina os estados que a pessoa precisa reconhecer e as ações que devem funcionar.

Tela ou estado	Ação e resposta observáveis
Lista vazia	Exibe uma orientação curta e o botão Nova nota com nome acessível.
Lista com dados	Exibe a ordem acordada. Toque em uma nota abre leitura. Voltar retorna à lista.
Editor	Campo rotulado, teclado acessível e botão Salvar. Texto em branco mantém o editor e mostra a orientação perto do campo.
Gravação	A ação fica temporariamente indisponível. A lista abre após confirmação da persistência.
Falha	Mostra um erro compreensível, preserva o rascunho e oferece nova tentativa quando a falha é recuperável.
Leitura	Mostra a nota persistida com texto selecionável se previsto pelo design. A tela oferece retorno à lista.

CHAT / ESPECIFICAÇÃO DA INTERFACE

```
$ui-design
Draft docs/ui-design.md for the agreed note journey.
Specify empty, editing, saving, success, and error states.
Cover focus, TalkBack labels, font scaling, and recovery.
Link the OpenSpec scenarios. Design only.
```

PROTÓTIPO OPCIONAL

Para decidir layout, use um experimento isolado com dados fictícios. Na passagem para o app, revise o código reaproveitado, substitua o armazenamento simulado e rode o fluxo na UI real.

04 / PRODUTO

Escolha uma arquitetura mínima

O laboratório usa um módulo app com limites internos claros. Separe módulos Gradle quando houver uma razão técnica para isso.

Tela Compose > **ViewModel + estado** > **Repositório de notas** > **DAO Room** > **Banco privado**

A tela envia ações e exibe estado. O ViewModel coordena a ação de salvar com escopo de ciclo de vida. O repositório traduz resultados do armazenamento. O DAO persiste e observa as notas. Use Flow para a lista e coleta compatível com o ciclo de vida na UI.

Escolha	Aplicação na primeira change
Kotlin e Compose	Use os plugins e versões compatíveis com o template. Centralize dependências no catálogo, se adotado.
Room e KSP	Use versões compatíveis com Kotlin e AGP. Configure exportação de schema e versione os JSONs gerados.
Injeção	Instancie o grafo manualmente neste app pequeno. Use android-di-koin se Koin for uma decisão do projeto.
Trabalho em segundo plano	A gravação local pertence à interação corrente. Use WorkManager para tarefas duráveis e adiáveis quando uma change futura precisar.

CRITÉRIO TÉCNICO

Proteja a gravação concorrente no caminho que de fato executa o efeito. Desabilitar o botão ajuda a experiência. O controle no ViewModel ou repositório impede duas chamadas simultâneas pelo fluxo.

Recomendações de arquitetura Android

05 / PRIMEIRA CHANGE

Crie a mudança de notas offline

Uma change é a unidade de trabalho. Dê um identificador específico e use a raiz do projeto que vai receber a implementação.

TERMINAL / ABRIR O DIRETÓRIO E CONSULTAR O SCHEMA

```
openspec new change add-offline-notes --schema spec-driven
openspec status --change add-offline-notes
openspec instructions proposal --change add-offline-notes
```

ARQUIVOS DO LABORATÓRIO

```
openspec/changes/add-offline-notes/
  .openspec.yaml
  proposal.md
  specs/offline-notes/spec.md
  design.md
  tasks.md
```

Use as instruções de cada artefato do schema instalado. No schema deste tutorial, a proposta orienta o delta e o design, que dão base às tarefas. As próximas cinco páginas mostram o conteúdo para copiar ou adaptar.

CHAT / ALTERNATIVA PARA CRIAR OS ARTEFATOS

```
$openspec-propose
Draft change add-offline-notes from the accepted product brief.
Use the spec-driven schema and the capability offline-notes.
Include persistence, validation, failure, and duplicate-submit cases.
Draft the artifacts only. List any decision needing my input.
```

ESCOLHA UM CAMINHO

Use o comando do terminal para escrever os arquivos com o guia, ou peça ao agente para criar a change e os artefatos. Se o diretório já existe, informe o ID e peça para trabalhar nele.

Schema padrão spec-driven

Ciclo de uma change

05 / PRIMEIRA CHANGE

1. Escreva a proposta

Conte por que a mudança existe e qual capacidade ela afeta. O revisor deve conseguir delimitar a entrega sem abrir o código.

OPENSPEC/CHANGES/ADD-OFFLINE-NOTES/PROPOSAL.MD

```
## Why

People need to capture text without connectivity and read it later.

## What Changes

- Create, list, and open local text notes.
- Validate input and recover from a failed save.
- Keep saved notes available after reopening the app.

## Capabilities

### New Capabilities

- `offline-notes`: Create and read notes stored on the device.

### Modified Capabilities

## Impact

- Android list, editor, and reader screens.
- Private Room database and persistence tests.
- Editing, deleting, accounts, sync, and sharing are out of scope.
```

Revise o recorte

01 **Cheque impacto e escopo.** O usuário ganha uma forma de salvar e ler texto offline. A proposta menciona as telas e o armazenamento tocados.

02 **Identifique a capacidade durável.** Use offline-notes para o comportamento que permanecerá depois de add-offline-notes ser arquivada.

RESULTADO ESPERADO

Uma decisão sobre sincronização, exclusão ou exportação que surgir aqui pode entrar em uma change futura com verificação própria.

05 / PRIMEIRA CHANGE

2. Defina o comportamento

Use requisitos que descrevam o que a pessoa observa e cenários que revelem regressões. Comece com a jornada e o primeiro caso negativo.

SPECS/OFFLINE-NOTES/SPEC.MD / PARTE 1 DE 2

```
## Purpose

Create and reopen local text notes without network access.

## ADDED Requirements

### Requirement: Save and reopen notes offline
The app SHALL save a non-blank note to local storage without
network access. It SHALL confirm success only after the write
commits. It SHALL list saved notes newest first with a stable
tie-break and allow opening a note with its original text.

#### Scenario: Note survives reopening
- **WHEN** a person saves "Comprar pão" in airplane mode, closes
  the app, and opens it again without clearing its data
- **THEN** the saved note is listed and opens with the same text

#### Scenario: New notes appear first
- **WHEN** a person successfully saves note A and then note B
- **THEN** both are listed, with B before A, including on reopening

### Requirement: Reject blank notes
The app SHALL reject empty and whitespace-only input without
creating a record. It SHALL keep the editor open with guidance.

#### Scenario: Whitespace-only input
- **WHEN** a person attempts to save spaces only
- **THEN** the editor explains the error and no note is stored
```

O comando SHALL expressa o que a aplicação precisa cumprir. WHEN descreve o gatilho. THEN descreve o efeito verificável. “Fechar” neste cenário significa finalizar o processo sem limpar os dados.

REVISÃO RÁPIDA

Se o teste só verifica que o botão responde, ele pode passar mesmo quando a nota some após fechar o app. O cenário exige armazenamento e leitura reais.

05 / PRIMEIRA CHANGE

2. Complete os casos de falha

Continue no mesmo spec.md. Guarde o contrato visível aqui e os detalhes de Room, Flow e tratamento de erros em design.md.

SPECS/OFFLINE-NOTES/SPEC.MD / PARTE 2 DE 2

```
### Requirement: Recover from a failed save
If a write fails, the app SHALL keep the existing saved notes
unchanged and retain the text in the active editor. It SHALL
report the failure and allow a retry for recoverable errors.

#### Scenario: Recoverable write failure and retry
- **WHEN** the first save fails before committing and a person
  retries successfully with the same draft
- **THEN** the error and draft are visible after the failure,
  and exactly one new note appears after retrying

### Requirement: Guard an in-flight save
The app SHALL accept at most one save action while that editor's
write is in progress.

#### Scenario: Repeated activation during a write
- **WHEN** a person activates Save twice before the first write ends
- **THEN** only one write is attempted and one note is saved
```

Questão	Decisão do laboratório
Quando a falha é testada?	Antes do commit, com uma dependência controlada que falha uma vez. Depois, libere a gravação e verifique um único registro.
Qual é a garantia da guarda?	Um único efeito enquanto a mesma gravação está em curso. Duplicidade entre sessões ou processos exigiria uma regra e mecanismo próprios.
O que acontece com o rascunho?	Permanece no editor ativo. Retomada de rascunho após morte do processo fica para outra change.

Use o título exato do cenário para ligar delta, tarefa, teste e evidência. Se renomear um cenário já consolidado, confira as regras do schema antes de arquivar.

05 / PRIMEIRA CHANGE

3. Registre o design

Registre as decisões técnicas, seus limites e o que muda quando uma premissa falha. Use o texto como ponto de partida.

OPENSPEC/CHANGES/ADD-OFFLINE-NOTES/DESIGN.MD

```
## Context

This is a single-module Kotlin app with Compose and local storage.

## Goals / Non-Goals

- Deliver the offline create, list, and read journey.
- Keep existing notes intact on failure.
- Sync, sharing, and draft restoration are out of scope.

## Decisions

- Use Room for private storage and export the version 1 schema.
- Use an auto-generated, increasing ID for newest-first ordering.
- Observe saved notes through the repository and ViewModel.
- Guard save actions before suspending and confirm only on commit.
- Map recoverable write errors to a retryable UI state.
- Exclude the database from backup and device transfer where
  supported by the configured Android rules.

## Risks / Trade-offs

- A fake cannot prove restart persistence; run the real DAO and UI.
- Missing draft restoration must be clear in the test scope.
- Treat schema upgrades as changes with migration evidence.
```

A chave gerada e ordenada por inserção evita depender da hora do aparelho para a ordem neste app. Se o app usar importação ou sincronização no futuro, especifique como a ordem e os IDs mudam.

DEPENDÊNCIAS

Registre Room, KSP, SDK e o runner escolhidos no projeto. Se uma versão for incompatível, corrija a matriz Gradle/JDK/Kotlin antes de criar uma segunda solução de armazenamento.

05 / PRIMEIRA CHANGE

4. Divida em tarefas verificáveis

Organize as tarefas para chegar cedo a uma jornada que funciona. Relacione cada tarefa a um comportamento ou evidência esperada.

OPENSPEC/CHANGES/ADD-OFFLINE-NOTES/TASKS.MD

```
## 1. Implement the offline journey

- [ ] 1.1 Wire list, editor, and reader through the ViewModel.
- [ ] 1.2 Persist with Room and export schema v1.
- [ ] 1.3 Verify Note survives reopening and New notes appear first.

## 2. Implement validation and recovery

- [ ] 2.1 Reject whitespace and guard an in-flight save.
- [ ] 2.2 Keep the draft after failure and support a safe retry.
- [ ] 2.3 Verify whitespace, repeated activation, and retry cases.

## 3. Verify the candidate

- [ ] 3.1 Run the agreed checks and link revision-specific proof.
- [ ] 3.2 Review the manifest, data rules, and accessibility.
```

01 **Primeira fatia.** UI, ViewModel e Room salvam e reabrem uma nota em modo avião. Faça o teste em um emulador ou aparelho de teste.

02 **Segunda fatia.** Adicione texto em branco, bloqueio de segundo acionamento e erro recuperável. Observe efeitos persistidos e estados da tela.

03 **Revisão da candidata.** Execute a matriz acordada, registre os resultados e reabra a tarefa correspondente quando surgir uma falha.

CHECKLIST ATIVO

Marque uma tarefa depois da evidência que ela pede. O arquivo tasks.md acompanha o trabalho. PROOF retém os comandos, resultados e limites por revisão.

05 / PRIMEIRA CHANGE

Valide e aceite o contrato

Valide a estrutura no CLI e faça uma leitura de conteúdo. Registre a aceitação e a autorização de implementar no fluxo do projeto.

TERMINAL / INSPECIONAR A CHANGE ANTES DE CODIFICAR

```
openspec status --change add-offline-notes --json
openspec validate add-offline-notes --strict
openspec instructions apply --change add-offline-notes --json
```

Sinal	Leitura correta
Artefatos prontos	Os arquivos requeridos foram reconhecidos pelo schema.
Validação estrita passa	O CLI aceitou a estrutura, o delta e as relações que ele verifica.
Tarefas 0 de N	A implementação e a verificação ainda estão pendentes.
Requisito aceito	O responsável confirmou o comportamento e os limites.

01 **Faça uma leitura humana.** Simule nota vazia, falha de escrita, toque repetido, fechamento e reabertura. Resolva as ambiguidades de produto.

02 **Identifique o próximo incremento.** Registre change, tarefas, revisão de referência, quem decide e limites de execução em PROCESS ou no PR.

03 **Dê a autorização da fatia.** A frase “implemente somente a seção 1” delimita a ação. Se surgir conflito, preserve a intenção e peça a decisão que falta.

CHECAGEM REPRODUZIDA

No ensaio com CLI 1.16.0, o planejamento ficou completo com 0 de 8 tarefas feitas. A validação estrita passou. Os resultados Android deste guia são critérios para executar no seu projeto.

06 / IMPLEMENTAÇÃO

Prepare arquivos e dependências

Estes caminhos são uma organização inicial para o módulo app. Use a estrutura que o template e o projeto realmente oferecem.

Onde	O que preparar
gradle/libs.versions.toml	Versões de plugins e bibliotecas quando o projeto usa catálogo. Preserve o padrão gerado pelo Studio.
app/build.gradle.kts	SDKs, variante, runner de instrumentação, dependências e geração de schema.
app/src/main/	Manifesto, recursos e código Kotlin. Separe pacotes ui e data. Use res/values/strings.xml para textos.
app/src/test/	Regras puras, ViewModel, corrotinas e fakes executados na JVM.
app/src/androidTest/	Testes com AndroidJUnitRunner, Compose, Room e migração em um dispositivo.
app/schemas/	Schemas exportados quando esse for o diretório configurado. Guarde as versões necessárias para testar upgrades.

01 **Ajuste a toolchain em uma etapa.** Para seguir o exemplo Room 2.x das skills, alinhe runtime, compilador via KSP, plugin Room e biblioteca de testes dessa linha. Configure schemaDirectory no módulo que possui a base. Em Room 3, confira coordenadas e APIs da linha escolhida.

02 **Rode uma compilação antes de ligar a UI.** Confira se o DAO foi gerado e se o JSON do schema aparece no diretório configurado. Ajuste o runner e os assets de schema conforme a versão adotada.

Room 2.x no guia de build da coleção

Diferenças entre Room 2 e Room 3

06 / IMPLEMENTAÇÃO

Implemente a primeira fatia

Comece pela jornada completa. Você deve conseguir criar, reabrir e ler uma nota com o app em modo avião.

CHAT / IMPLEMENTAÇÃO DELIMITADA

```
$software-delivery
Implement only section 1 of accepted change add-offline-notes.
Use the existing Compose app and the approved Room design.
Trace each task to its OpenSpec scenario.
Run the relevant Android checks and report observed results.
Record pending checks and stop after this increment.
```

01 **Use a base que abriu no emulador.** Crie a branch de trabalho conforme a política do repositório. Confira status, revisão, módulo e ambiente antes de editar.

02 **Implemente dados e integração.** Use android-data-layer para entidade, DAO e repositório. Versione o schema v1 de Room e verifique o caminho real da gravação. Use android-module-structure se o build ou os módulos precisarem mudar.

03 **Ligue a interface.** Use ui-design para Compose e navegação. Encaminhe eventos ao ViewModel e observe dados do Room. A confirmação de sucesso vem do resultado de gravação.

04 **Execute o primeiro cenário.** Desative a rede no dispositivo descartável, salve um texto fictício, force o encerramento sem limpar dados, relance e abra a nota. Confira o texto e a ordem.

ANTES DE MARCAR A TAREFA

Uma UI com lista em memória é útil para explorar layout. Feche a primeira fatia somente depois de ver o texto reaparecer em uma nova execução com a base real.

06 / IMPLEMENTAÇÃO

Proteja a gravação e a recuperação

Na segunda fatia, demonstre que a aplicação preserva dados existentes e oferece recuperação compreensível.

CHAT / VALIDAR FALHAS E EFEITOS

```
$software-delivery
Implement section 2 of add-offline-notes.
Preserve the accepted behavior and existing tests.
Exercise blank input, a controlled write failure, and
two activations while the first save remains in flight.
Assert the persisted row count as well as the UI state.
```

01 **Valide antes da escrita.** Teste string vazia e texto composto só de espaços. Nenhuma linha deve ser criada. Em caso válido, preserve o texto digitado.

02 **Feche a janela de corrida.** Bloqueie a ação antes do primeiro ponto de suspensão. Use um fake que segure a primeira chamada até o teste acionar a segunda. Verifique o contador e o efeito após liberar.

03 **Controle a falha.** Faça a primeira gravação falhar antes de persistir. Confirme erro e rascunho na UI, notas anteriores intactas e nenhuma inserção parcial. Libere o retry e comprove uma nova linha.

04 **Respeite o ciclo de vida.** Trate erros esperados no limite do armazenamento. Propague cancelamento e preserve a política de navegação. Use android-error-handling quando precisar padronizar o resultado e a recuperação.

SE O COMPORTAMENTO MUDAR

Atualize o delta, design, tarefas e cenário afetado quando a decisão de produto for revista. Registre quem aprovou e refaça os checks da fronteira tocada.

07 / VERIFICAÇÃO

Monte a matriz de testes

A evidência mais útil entra pela fronteira que pode falhar. Use casos positivos e negativos, incluindo efeitos persistidos.

Cenário OpenSpec	Teste principal	Evidência que importa
Note survives reopening	Integração Room e roteiro no app real.	Após reiniciar o processo, o texto aparece e abre igual.
New notes appear first	DAO ou integração com dois registros.	Ordem B, A no resultado e na UI após reabertura.
Whitespace-only input	Unitário e UI instrumentada.	Mensagem visível, editor aberto e zero gravações.
Recoverable write failure and retry	Fake controlado no ViewModel e jornada instrumentada, conforme a estrutura.	Rascunho preservado, dados prévios iguais e uma inserção após retry.
Repeated activation during a write	Unitário com suspensão controlada.	Uma tentativa enquanto a primeira chamada está em curso.

01 **Use unitários para regras e concorrência controlada.** Peça android-unit-testing para a lógica que roda na JVM.

02 **Use integração para armazenamento.** Peça android-integration-testing para DAO real, schema e migração.

03 **Use UI instrumentada para interação.** Peça android-ui-testing para semântica, navegação e estados visíveis. Emulador e aparelho complementam os testes locais.

Fundamentos de testes Android

07 / VERIFICAÇÃO

Rode os checks no ambiente certo

Exemplos para :app e variante debug, sem flavors. Consulte :app:tasks --all para o nome real das tarefas.

TERMINAL / BUILD, JVM, LINT E TESTE INSTRUMENTADO

```
./gradlew :app:assembleDebug :app:testDebugUnitTest :app:lintDebug
adb devices -l
./gradlew :app:connectedDebugAndroidTest
```

01 **Escolha o alvo.** Deixe conectado apenas o AVD ou aparelho previsto para a rodada de instrumentação. Registre API, ABI, build e se era um dispositivo físico.

02 **Verifique execução real.** Confira contagem de testes, pulados e falhos, relatórios e logs. Um teste unitário com fake comprova reação ao erro controlado. Execute Room e o app para persistência.

03 **Teste uma falha representativa.** Quando possível, confirme que a asserção falha contra a versão defeituosa em uma cópia descartável. Restaure a implementação correta e confirme que o mesmo teste passa.

04 **Repita nas variantes relevantes.** Rode a matriz no mínimo suportado e em uma API recente. Confira artefato release separadamente. No aparelho físico, exercite a jornada principal e acessibilidade.

QUANDO UM CHECK FALHA

Leia a primeira causa, confirme a toolchain e reproduza o menor cenário. Use diagnose-defect se a causa seguir incerta. Preserve a asserção até entender a falha. Registre bloqueios e checks pendentes.

07 / VERIFICAÇÃO

Execute a jornada como usuário

Use dados fictícios, ambiente descartável e o build que está sendo avaliado. Repita com fonte ampliada e TalkBack quando aplicável.

01 **Primeira abertura.** Em instalação limpa de teste, abra a lista. Confira estado vazio, foco e rótulo de Nova nota. Ative o modo avião pelo dispositivo.

02 **Salve duas notas.** Crie “Comprar pão” e “Ligar às 17h”. Confira a ordem, abra cada texto e volte à lista. Tente salvar espaços em branco e confirme que a lista ficou igual.

03 **Reabra o processo.** Finalize o app no dispositivo sem limpar dados. Inicie pelo launcher e confira as duas notas. Rotacione o aparelho e avalie o comportamento previsto para cada tela.

04 **Confira acessibilidade.** Com TalkBack, navegue em ordem, abra o editor, identifique o rótulo do campo, acione Salvar e confira o anúncio da mensagem de erro. Com fonte ampliada, observe corte, rolagem e alcance das ações.

TERMINAL / ENCERRAR APENAS O APP DIDÁTICO NO AVD

```
adb devices -l
adb -s emulator-5554 shell am force-stop com.example.notasdebolso
```

Substitua o serial pelo exibido no seu terminal e confirme o applicationId real da variante. Reabra o app pelo launcher. Use falha de escrita injetada pelo teste para o cenário de recuperação.

Testes de acessibilidade com Compose e TalkBack

07 / VERIFICAÇÃO

Guarde a evidência por revisão

Registre resultados em docs/proof.md ou no local adotado pelo projeto. Uma pessoa que abre o PR deve conseguir repetir os checks.

DOCS/PROOF.MD / MODELO PARA PREENCHER

```
## Evidence for add-offline-notes

- Revision: <commit or immutable candidate identifier>
- Variant and environment: <debug, API, ABI, device>
- Toolchain: <JDK, SDK, Gradle, OpenSpec versions>
- Scenario: Note survives reopening
- Check: <command or manual sequence>
- Observed: <result, expected vs actual, report path>
- Status: <passed | failed | not run | blocked>
- Limit: <boundary not exercised>
- Follow-up: <owner and next action>
```

Registro	Como usar
Relatório de teste	Vincule a execução, teste, contagem e versão do código. Anote quando o teste usou fake ou armazenamento real.
Evidência manual	Inclua passos, aparelho/API, versão do app e resultado. Use capturas sem dados pessoais quando esclarecerem o cenário.
Pendência	Indique ambiente faltante, impacto, dono e o que libera a próxima etapa.

FRESCOR DA EVIDÊNCIA

Se a implementação ou as regras mudarem, mantenha a evidência antiga ligada ao snapshot que ela verificou. Rode novamente os checks afetados para a versão que vai integrar ou publicar.

08 / REVISÃO

Revise segurança e privacidade

No laboratório, o maior risco é perder ou expor o conteúdo de uma nota. Revise controles com os artefatos e variantes reais.

Fronteira	O que conferir e provar
Armazenamento	Dados no sandbox do app. Abra o manifesto mesclado e confira as regras de backup e transferência para as APIs e dispositivos suportados.
Permissões	Confirme a necessidade de cada permissão. Compare debug e release. Neste piloto, a jornada usa zero chamadas de rede e funciona em modo avião.
Componentes	O launcher pode ser exportado. Revise activities, providers, receivers e services adicionais, além de acesso por intents, URIs e deep links, se houver.
Dependências	Confira origem, compatibilidade, licenças e efeitos. Um SDK pode adicionar permissões ou código nativo por transitividade.
Logs e segredos	Procure conteúdo de notas, chaves ou dados de assinatura no diff, Logcat e relatórios. Use dados fictícios e mantenha a chave fora do repositório.

CHAT / AUDITORIA SOMENTE LEITURA

```
$cybersec
Review the Android release candidate for data exposure and loss.
Inspect the merged manifest, data rules, logs, and dependencies.
Cite observable evidence and limits. Keep this review read-only.
```

Para autenticação futura, teste sucesso e negação atravessando o ponto que realmente impede acesso. Inspecione efeitos no estado. Caso a permissão esteja no servidor, o teste precisa atingir o servidor de teste.

08 / REVISÃO

Integre com Git e CI

Use uma branch curta ou o fluxo definido pela equipe. O PR vincula contrato, implementação e evidência da revisão candidata.

01 **Prepare a CI.** No GitHub Actions, use o mesmo JDK, wrapper e SDK do projeto. Em PRs, execute build, JVM, lint e validação de specs quando o CLI estiver configurado. Guarde relatórios e use permissões mínimas.

02 **Decida onde rodam testes de dispositivo.** Comece com execução local registrada. Se a equipe tiver runner e emulador confiáveis, automatize a matriz que já funciona. Registre skip e falta de alvo explicitamente.

03 **Revise o PR.** Inclua change, resumo, cenários, riscos, evidência, checks pendentes e revisão. Uma falha obrigatória bloqueia integração até correção ou decisão formal da política.

04 **Use git-delivery para publicar a revisão.** Confirme o destino, escopo, autorizações, hooks e ações disparadas. Depois do push, confira o commit remoto e o resultado da CI para aquele hash.

TERMINAL / CONFERIR A CANDIDATA

```
git status --short --branch
git diff --check
git diff --stat
git log -1 --oneline
```

O QUE VERDE SIGNIFICA

Um PR com build verde confirma o que o workflow realmente executou. Vincule o resultado de aparelho físico, TalkBack, migração e trilha da Play quando esses checks ocorrerem fora da CI.

ANDROID / REVISÃO E CONSOLIDAÇÃO

Consolide a primeira capacidade

Aplique a revisão comum das páginas 17 e 18 ao ID add-offline-notes. Antes do upgrade, deixe o comportamento v1 consolidado e preserve um APK v1 e uma base de teste com notas.

```
$peer-review
Review the exact candidate diff for add-offline-notes.
Read the accepted delta and docs/proof.md.
Trace persistence, failed writes, and repeated activation.
Show how the tests expose each defect. Read only.
```

Informe base, revisão, variante, ambiente, comandos e pendências. Use contexto separado nos riscos de perda de dados e mudanças de schema. Trate cada achado com gatilho, efeito, localização e check que discrimine a correção. Após a correção, repita os checks afetados. O mantenedor decide a integração.

Após concluir tarefas e verificações pertinentes

```
openspec validate add-offline-notes --strict
openspec instructions apply --change add-offline-notes --json
openspec archive add-offline-notes
openspec list --archived
openspec list --specs
openspec validate --all --strict
git status --short
git diff -- openspec
```

Confirme a autoridade para arquivar e use a revisão de evidência que satisfaz os cenários. Inspecione openspec/specs/offline-notes/spec.md e o histórico. Complete um Purpose provisório e revalide. Aponte PRODUCT para a capacidade consolidada e mantenha os resultados em PROOF.

Condições para o próximo laboratório

- Gere um APK v1 identificado e preserve o schema exportado. Instale-o no dispositivo descartável e crie notas distintas com dados fictícios.
- Registre applicationId, assinatura, variante, versionCode e hash. Use-os ao selecionar o APK v2 compatível para atualizar sobre a instalação v1.
- Preserve dados e a instalação ao executar o upgrade. Registre a revisão do código, do contrato e dos artefatos no resultado de teste.

09 / LABORATÓRIO DE UPGRADE

Segunda change: fixe notas

Agora existe uma base instalada. Exercite a diferença entre começar do zero e preservar dados de quem já usa o app.

01 **Guarde uma versão v1.** Arquive o APK de teste, hash, applicationId, assinatura de teste, versão de schema e commit. No dispositivo descartável, crie duas notas na v1 e anote texto e ordem.

02 **Proponha pin-offline-notes.** Use a capacidade offline-notes existente. O usuário pode fixar e desafixar. Notas fixadas vêm primeiro. Dentro de cada grupo, conserve a ordem por inserção mais recente. Notas antigas começam desafixadas.

03 **Planeje e teste a migração.** No design, acrescente uma coluna pinned com default 0 em uma migração Room v1 → v2. Alinhe o default da entidade com o schema migrado e teste a abertura do banco preexistente. Preserve textos e IDs.

04 **Verifique os dois caminhos.** Use android-integration-testing com os schemas v1 e v2 e dados semeados. Em seguida, atualize o app real sem limpar dados e repita a jornada. Teste instalação limpa separadamente.

CHAT / ESPECIFICAÇÃO DE UMA ALTERAÇÃO EM APP EXISTENTE

```
$openspec-propose
Draft pin-offline-notes for existing capability offline-notes.
Preserve the existing scenarios in modified requirements.
Add pin and unpin behavior and a v1-to-v2 upgrade scenario.
Describe a non-destructive Room migration and its verification.
Draft only; ask about material ambiguities.
```

CUIDADO COM O DELTA

Em MODIFIED Requirements, reproduza o bloco completo de cada requisito alterado e mantenha os cenários que continuam válidos. A validação estrita pode apontar cenário omitido. O arquivo arquivado é a referência da versão anterior.

ANDROID / UPGRADE

Atualize a capacidade existente

Salve esta parte e a da página seguinte, nessa ordem, no mesmo arquivo openspec/changes/pin-offline-notes/specs/offline-notes/spec.md. A spec consolidada da v1 é a referência para os cenários preservados.

```
## MODIFIED Requirements

### Requirement: Save and reopen notes offline
The app SHALL save a non-blank note to local storage without
network access. It SHALL confirm success only after the write
commits. It SHALL list pinned notes before unpinned notes.
Within each group it SHALL list the newest notes first using a
stable tie-break. It SHALL open saved notes with their original text.

#### Scenario: Note survives reopening
- **WHEN** a person saves "Comprar pão" in airplane mode, closes
  the app, and opens it again without clearing its data
- **THEN** the saved note is listed and opens with the same text

#### Scenario: New notes appear first
- **WHEN** a person successfully saves note A and then note B
- **THEN** both are unpinned and B appears before A, including
  on reopening
```

Planeje a migração

Em proposal.md, vincule offline-notes em Modified Capabilities. Em design.md, registre a coluna pinned, o default 0, o caminho de v1 a v2 e as consultas de ordenação. Alinhe o default da entidade com o schema migrado. Mantenha as decisões de backup e privacidade nos registros que as governam.

Em tasks.md, vincule migração, fixar/desafixar, instalação limpa e atualização real a verificações próprias. Teste o banco v1 com linhas conhecidas e valide schema e conteúdo. Na UI, confira leitura, ordem e persistência depois de reabrir.

Android Developers · migrações do Room

ANDROID / UPGRADE

Comprove fixação e dados preservados

Acrescente este bloco depois de MODIFIED Requirements no delta pin-offline-notes. Mantenha a primeira change arquivada e trabalhe sobre a capacidade consolidada.

```
## ADDED Requirements

### Requirement: Pin and unpin notes
The app SHALL allow changing the pin state of a saved note.
New and existing v1 notes SHALL start unpinned.

#### Scenario: Pin state survives reopening
- **WHEN** a person pins an older note and reopens the app
- **THEN** it remains pinned and appears before unpinned notes

#### Scenario: Unpin restores insertion order
- **WHEN** a person unpins that note and reopens the app
- **THEN** it is unpinned and returns to its insertion order
  among unpinned notes

#### Scenario: Upgrade preserves notes
- **WHEN** an installation with v1 notes upgrades to v2
- **THEN** existing notes retain their IDs and texts and
  start unpinned
```

```
openspec show offline-notes --type spec
openspec validate pin-offline-notes --type change --strict
openspec instructions apply --change pin-offline-notes --json
```

Faça a prova de upgrade com o mesmo applicationId, chave de teste e dados da v1. A v2 precisa de versionCode maior. Teste tanto o APK instalado quanto a integração da migração em Room. Rode instalação limpa em outro alvo descartável. Registre cada prova com seus próprios dados e artefato.

Uma validação estrita pode rejeitar a omissão de um cenário que o MODIFIED deve preservar. Use os títulos originais no delta e teste a nova expectativa no app.

09 / LABORATÓRIO DE UPGRADE

Teste a atualização no dispositivo

Use dois APKs da mesma identidade e da mesma assinatura de teste. A versão candidata deve ter versionCode maior.

TERMINAL / NO EMULADOR DESCARTÁVEL, COM APKS DE TESTE

```
adb devices -l
adb -s emulator-5554 install /caminho/v1.apk
# Abra v1 e crie duas notas. Preserve os dados para a troca.
adb -s emulator-5554 install -r /caminho/v2.apk
# Abra v2. Verifique notas antigas, pin, unpin e reabertura.
```

01 **Valide a identidade antes de instalar.** Confira serial, package, assinatura, códigos de versão e hash. Use arquivos exportados da variante escolhida.

02 **Evite um falso positivo.** Se desinstalar, trocar o applicationId ou limpar dados, você executa outro cenário. Restaure o AVD descartável à v1, semeie as notas e repita o upgrade.

03 **Reproduza uma falha de migração.** Em uma cópia isolada, verifique se o teste detecta default incompatível ou perda de linha. Corrija e repita o caminho completo com a base original.

04 **Verifique a próxima versão separadamente.** Para distribuição pela Play, valide instalação e upgrade pela trilha real. A identidade de assinatura da Play difere da chave de teste local.

RECUPERAÇÃO

Planeje uma correção que leia o schema já aplicado. Uma versão antiga pode recusar uma base atualizada. O plano de recuperação precisa proteger dados existentes.

10 / DISTRIBUIÇÃO

Prepare uma candidata Android

Um APK é instalável. Um AAB é o pacote usado pela Play para gerar APKs para o aparelho. Registre qual artefato e qual revisão foram testados.

TERMINAL / EXEMPLOS PARA A VARIANTE RELEASE

```
./gradlew :app:lintRelease :app:bundleRelease
git rev-parse HEAD
# Ajuste o caminho ao artefato que o Gradle gerou.
sha256sum app/build/outputs/bundle/release/app-release.aab
```

01 **Inspecione a variante.** Confira applicationId, versionCode, versionName, manifesto mesclado, assets, minSdk, targetSdk, dependências, regras de otimização e avisos. Teste o release real, inclusive se houver R8.

02 **Valide o bundle.** Use bundletool validate, gere um conjunto de APKs com build-apks e instale com install-apks em aparelho de teste. Siga a documentação para a assinatura de teste. Se houver bibliotecas nativas próprias ou de terceiros, avalie compatibilidade de ABI e 16 KB quando aplicável.

03 **Configure a assinatura para publicação.** Use o Android Studio em Build → Generate Signed Bundle / APK ou um pipeline aprovado. A chave de upload assina o bundle enviado. O Play App Signing usa a chave de assinatura do app para os APKs distribuídos.

04 **Proteja os materiais.** Restrinja acesso e guarde chave, senha e recuperação fora do Git. Registre somente identificadores, hashes e o local do procedimento de acesso.

Assinatura Android e Play App Signing

Validação e geração de APKs com bundletool

Compatibilidade com páginas de memória de 16 KB

10 / DISTRIBUIÇÃO

Experimente o bundle localmente

Use bundletool para instalar em um emulador descartável os APKs gerados da candidata. Isto exercita o empacotamento antes da trilha.

TERMINAL / AJUSTE OS CAMINHOS E O SERIAL

```
BUNDLE=app/build/outputs/bundle/release/app-release.aab
SERIAL=emulator-5554
adb devices -l
bundletool validate --bundle="$BUNDLE"
bundletool build-apks --bundle="$BUNDLE" --output=candidate.apks \
  --connected-device --device-id="$SERIAL"
bundletool install-apks --apks=candidate.apks --device-id="$SERIAL"
```

01 **Prepare bundletool.** Instale o binário ou use java -jar seguido do caminho para o JAR oficial. Confira a versão e confirme que o dispositivo é o esperado.

02 **Use assinatura de teste.** Sem configuração de chave, o bundletool tenta usar uma chave de debug. Para ensaiar atualização local, use a mesma identidade e a mesma assinatura nos APKs v1 e v2. Faça a instalação em um alvo que pode ser descartado.

03 **Registre o que rodou.** Abra o app, confirme versionCode, jornada offline, leitura e estado após reabertura. Guarde hash do AAB, versão do bundletool, serial de teste redigido se necessário e resultados.

PRÓXIMO CHECK

Repita na trilha da Play para verificar a identidade de assinatura, os APKs servidos e o upgrade no canal real.

Referência oficial de bundletool

10 / DISTRIBUIÇÃO

Teste em uma trilha da Play

Esta etapa precisa de um app no Play Console e uma conta com acesso. Comece com um grupo controlado e um artefato rastreável.

01 **Configure a ficha do app.** Use applicationId definitivo e confirme políticas aplicáveis, público-alvo, declarações, classificação, privacidade, Data safety e requisitos de target API. Verifique também SDKs de terceiros.

02 **Suba a candidata à trilha escolhida.** Para um ensaio inicial, use teste interno conforme a disponibilidade da conta. Associe o AAB assinado, notas de versão e testadores. Registre a trilha e o versionCode.

03 **Instale pelo link da trilha.** Com conta de testador autorizada, aceite o convite e instale pela Play. Confira o versionCode realmente servido no aparelho. Em uma segunda versão, atualize pela mesma trilha com dados de teste existentes.

04 **Siga os requisitos da sua conta.** A Play define requisitos adicionais para determinadas contas pessoais novas. Consulte o painel e a documentação antes de planejar a data de publicação.

05 **Decida promover.** Revise checks da trilha, acessibilidade, perda de dados, direitos de assets, observações pré-lançamento e capacidade de recuperação. Faça a promoção com autorização explícita.

VERIFICAÇÃO EM DUAS PONTAS

A aprovação do upload confirma o processamento do bundle. Registre também qual versão os testadores instalaram, se o upgrade preservou as notas e se a jornada passou.

Teste de contas pessoais novas

Formulário Data safety

Requisitos vigentes de target API

10 / DISTRIBUIÇÃO

Acompanhe e saiba recuperar

Documente em PRODUCTION quem observa a entrega, como confirma a versão em uso e o que faz quando uma checagem falha.

Sinal	Ação e evidência
Upload processado	Registre hash, revisão, trilha e versionCode. Confirme instalação no aparelho da trilha.
Candidata saudável	Exercite criação, reabertura, atualização de v1 e um cenário negativo. Anote aparelho, versão servida e resultado.
Falha de migração	Interrompa promoção. Preserve amostras e logs redigidos, identifique versões afetadas e prepare uma correção compatível com os schemas já instalados.
Aumento de crashes ou ANRs	Use relatórios da Play quando disponíveis. Segmente por versão, aparelho e janela. Compare com a linha de base e atribua um responsável pela resposta.
Usuários em rollout	Se a plataforma permitir, pause a ampliação da versão afetada. Avalie quem já recebeu o app e verifique o caminho de correção com versionCode superior.

01 **Defina a janela de observação.** Escolha sinais e critério de parada antes da promoção. Use feedback dos testadores e evidência compatível com a política de privacidade.

02 **Transforme incidentes em regressões.** Use diagnose-defect para isolar a causa e criar uma checagem que falha na versão problemática. Rode o controle na correção.

QUANDO HOUVER BACKEND

Use production-operations para planejar a homologação, verificar revisão servida e efeitos e ensaiar recuperação. O exemplo Notas de Bolso usa distribuição Android e armazenamento local.

LABORATÓRIOS C A F / AMBIENTE

Prepare os projetos de exercício

Use um diretório descartável para cada laboratório e rode os comandos na raiz indicada. Os geradores abaixo fornecem uma base. Depois do primeiro smoke, mantenha a toolchain e o lockfile registrados para repetir o trabalho em outro clone.

Web com React e TypeScript

```
npm create vite@latest receipt-portal -- --template react-ts
cd receipt-portal
npm install
npm run dev
# Em outro momento, confira o artefato que será entregue.
npm run build
npm run preview
```

Abra a URL impressa pelo servidor e faça um smoke. No projeto versionado, use npm ci para reproduzir as dependências. Quando configurar o Playwright, ligue o webServer ou a URL de teste ao ambiente isolado e confira que os dois contextos usam identidades distintas.

Desktop com Electron Forge

```
npx create-electron-app@latest desktop-notes --template=vite-typescript
cd desktop-notes
npm start
# No SO com o maker e suas dependências configurados:
npm run make
```

Abra a janela, registre o caminho de saída do maker e instale a primeira candidata na máquina alvo. Depois do smoke, use o preflight e o roteiro comum de instalação das skills e OpenSpec.

Serviço local, ferramenta MCP e fila

Registre os comandos do framework para iniciar dependências e o serviço. Faça uma chamada HTTP com fixture ou descubra uma ferramenta MCP local com o SDK selecionado. Para o worker, publique uma mensagem sintética e confirme uma saída no armazenamento descartável. Guarde as portas, modo de inicialização e comando de reset. Em MCP, confira no Inspector o transporte e a compatibilidade com o protocolo antes de usá-lo como prova.

Vite · base React e TypeScript

Electron Forge · geração e makers

MCP Inspector · configuração e execução

WEB / DO PROTÓTIPO À CHANGE

Monte um portal de recibos

Laboratório C. Uma pessoa autenticada consulta seus recibos e abre um detalhe. Use a API do laboratório B ou uma implementação local compatível com duas organizações de teste. Os recibos e credenciais devem ser fictícios.

1. Faça a jornada caber em um protótipo

Crie uma rota de lista e outra de detalhe no framework web escolhido. Peça a $ui-design dois modos de exibir lista vazia, carregamento e falha. Use objetos em memória e observe com teclado, viewport pequeno e zoom. Registre a opção escolhida, o que ficou sem validação e a condição de promoção.

2. Prepare a base executável

Em um projeto TypeScript com scripts configurados, rode npm ci, npm run dev e abra a URL informada pelo servidor. Use lockfile e fixe Node e dependências. Configure uma API descartável e contas A e B por fixtures. Ao escolher SSR, defina o que roda no servidor e o que chega ao navegador. Para uma SPA, verifique navegação direta e recarregamento de rotas no host de preview.

3. Especifique com PentaPRO e OpenSpec

- PRODUCT descreve a jornada, quem tem acesso e o que fica para outra change. PROJECT vincula OpenAPI ou contrato HTTP, identidade e política de cache. PROOF mapeia casos de acesso, falha e recuperação. PRODUCTION define preview e promoção. PROCESS aponta para a change e a ação autorizada.
- Abra view-tenant-receipts com o fluxo da página 10. Em proposal.md, inclua lista e detalhe somente leitura. Em design.md, prefira a mesma origem se possível, derive tenant da sessão, e valide acesso no backend. Para origens distintas, declare regras de CORS e cookie ou token em conjunto com $cybersec.
- Em tasks.md, ligue T1 a lista e detalhe com A, T2 ao isolamento A/B e expiração, T3 a estados de erro e acessibilidade, T4 ao build verificado no preview. Cada tarefa aponta para cenário e check.

```
$openspec-propose
Draft view-tenant-receipts for a read-only browser journey.
Use the existing HTTP contract and two isolated tenant fixtures.
Cover direct links, denied reads, expiry, and retry.
Draft only. Link the UI states and verification tasks.
```

Playwright · autenticação e isolamento

MDN · controles de origem no navegador

WEB / CONTRATO E EXECUÇÃO

Exercite a jornada no navegador

Salve este recorte em specs/receipt-portal/spec.md dentro da change. Registre os estados de carregamento, lista vazia, falha com retry, sessão expirada e acesso negado no documento de UI.

```
## Purpose

Let authenticated users view receipts from their own tenant.

## ADDED Requirements

### Requirement: Read receipts within the active tenant
The portal SHALL expose receipt details only to authorized users
of the owning tenant and provide a safe retry after a read failure.

#### Scenario: Authorized receipt opens after reload
- **WHEN** a user of tenant A opens a receipt of A by direct URL
  and reloads with the same valid session
- **THEN** the same receipt detail is visible

#### Scenario: Another tenant cannot read a receipt
- **WHEN** a user of tenant B opens the direct URL of A's receipt
- **THEN** access is denied and the response contains no A data

#### Scenario: Read recovers after a transient failure
- **WHEN** the API fails once and the user selects Retry
- **THEN** the page shows the authorized receipt after recovery
```

4. Implemente e prove a fronteira

- Use $software-delivery e $ui-design para a jornada. Use $http-api somente se mudar o servidor. A autorização deve ser aplicada no servidor, inclusive quando o ID chega por URL. Isole caches e resultados por identidade quando houver personalização.
- Rode openspec validate view-tenant-receipts --strict, o build e os checks do repositório. Em um projeto que configurou Playwright, rode npx playwright test. Use contextos separados para A e B e mantenha arquivos de sessão fora do Git. Use o servidor real na prova de acesso e falha controlada na prova de retry.
- Teste expiração no meio da navegação e logout seguido de nova sessão. Observe corpo e rede no acesso negado, foco ao mudar de rota e operação por teclado. Inclua teste com zoom e nos navegadores suportados.

5. Entregue o build observado

Com $production-operations, promova o artefato verificado. Confira rotas, cache, sessão e link direto no domínio alvo. Registre a revisão servida e a recuperação compatível com a API.

DESKTOP / ESCOPO E BUILD

Faça um editor de notas instalável

Laboratório D. Use Electron com TypeScript como exemplo de uma interface compartilhada. Escolha Qt ou APIs nativas quando a integração de SO ou a política do produto justificar. Declare Windows, macOS ou Linux e a arquitetura que você realmente pode exercitar.

1. Explore sem acesso a arquivos

Simule abrir e salvar com dados em memória. Avalie diálogo, atalho, fechamento com rascunho e feedback de erro. Use $software-delivery e $behavior-testing para promover o caminho escolhido. As rotinas web de UI Design podem ajudar no renderer. Os controles nativos exigem verificação própria.

2. Prepare uma base por SO

Inicialize um projeto pelo template Electron Forge escolhido. Fixe a toolchain no lockfile. Com os scripts padrão configurados, rode npm ci, npm start e npm run make. Instale ou abra o artefato produzido na máquina alvo. Documente dependências do maker, SO, arquitetura e localização da saída.

3. Crie save-desktop-notes

- Proposta. Salvar e reabrir uma nota no perfil local. Exportação e sincronização ficam para mudanças próprias. Contrato de armazenamento inclui formato, ordenação, rascunho em caso de falha e compatibilidade com perfil v1.
- Design. Processo principal controla o disco. Preload oferece apenas operações tipadas por nota. Renderer isolado, sandbox e integração Node desabilitada. Valide remetente e argumentos do IPC. Derive o caminho sob o perfil autorizado no processo principal. Use SQLite transacional ou substituição segura de arquivo, com garantias ajustadas ao SO e ao sistema de arquivos.
- Tarefas. T1 abre pacote e salva com armazenamento real. T2 valida entrada e testa falha de commit e IPC indevido. T3 faz upgrade sobre instalação v1 com notas. T4 empacota por SO e verifica assinatura, privacidade e recuperação.

```
$openspec-propose
Draft save-desktop-notes for a single local user profile.
Cover restart, failed writes, and upgrade from the shipped version.
Keep filesystem authority in the main process.
Draft artifacts only. Link each check to its target OS.
```

Electron · segurança, IPC e isolamento

Electron · empacotamento e distribuição

DESKTOP / COMPATIBILIDADE

Prove instalação, falha e upgrade

Salve em specs/desktop-notes/spec.md. Verifique a primeira nota no binário instalado e reserve uma base v1 para o upgrade com a v2 candidata.

```
## Purpose

Store and reopen notes inside a supported desktop user profile.

## ADDED Requirements

### Requirement: Persist notes within the active local profile
The editor SHALL preserve committed notes across restarts and
supported upgrades. On write failure it SHALL retain the draft
and the previous committed version.

#### Scenario: Installed editor reopens saved notes
- **WHEN** a user saves a note and restarts the installed editor
- **THEN** the editor opens the same content in that profile

#### Scenario: Failed write preserves saved content
- **WHEN** storage rejects a save before it commits
- **THEN** the draft remains editable and the old note is unchanged

#### Scenario: Installer upgrades a populated profile
- **WHEN** a supported v2 installer updates v1 with saved notes
- **THEN** the saved notes remain readable after restart
```

4. Verifique as fronteiras

- Rode openspec validate save-desktop-notes --strict, checks de domínio e integração IPC. Use perfil temporário com dados da v1, negue acesso ou injete falha na fronteira do disco e inspecione a nota após reabrir. Envie um IPC indevido e confira ausência de leitura ou escrita fora do perfil.
- Com $peer-review, cheque isolamento do renderer, validação de remetente, atomicidade e formato. Exercite navegação por teclado, atalhos, escala, foco e tecnologia assistiva no SO contratado.

5. Publique e atualize pelo canal de teste

Gere e identifique os instaladores por SO/arquitetura. Verifique assinatura e, quando aplicável, notarização no macOS. Valide upgrade de uma cópia instalada pelo mesmo canal. Para atualização automática, confira identidade, integridade, canal e falha de download. Planeje restauração de dados ou correção adiante quando o esquema já tiver migrado.

API / SERVIÇO NOVO

Abra a primeira fronteira HTTP

O laboratório B ensina a alterar uma API de recibos existente. Para construir do zero, comece por uma operação com cliente e banco reais. Uma boa primeira fatia é criar um recibo autenticado e poder consultá-lo pelo ID retornado.

1. Defina os consumidores e o contrato

Use $penta-pro para explicitar o principal, o tenant, os campos, limites e efeitos. Em OpenAPI ou contrato equivalente, registre métodos, paths, status, schema, erros e autenticação. Em OpenSpec, escreva cenários observáveis e vincule o contrato HTTP. Combine com o cliente o significado de um conflito de chave idempotente e a janela de retenção.

2. Implemente e acompanhe uma transação

- Com $http-api, valide o principal, derive o tenant no servidor, valide a entrada e coordene repositório e transação. Use chave de idempotência por tenant e operação quando houver retries com escrita. Garanta persistência atômica entre a decisão de deduplicar e o efeito prometido.
- Em $software-delivery, divida em iniciar API e banco com migração, criar e ler como A, negar acesso a B, testar retry e concorrência, e entregar ao preview. Comece com a rota mais curta que toca a autenticação, o banco e a resposta.

3. Teste cliente e estado juntos

```
# Em ambiente de teste, com URL e token sintéticos no shell.
curl -i -X POST "$API_URL/receipts" \
  -H "Authorization: Bearer $TEST_TOKEN_A" \
  -H 'Content-Type: application/json' \
  -H 'Idempotency-Key: lab-receipt-01' \
  --data '{"amountCents":1250,"currency":"BRL"}'
```

Este payload pressupõe esses campos no contrato do seu laboratório. Use uma fixture nova a cada rodada ou zere o estado pelo procedimento de teste. Compare status e corpo conforme o contrato, consulte o ID com A e tente com B. Repita a mesma chave com o mesmo corpo, com corpo diferente e com requisições simultâneas. Confira a quantidade de gravações duráveis e a ausência de dados de A na resposta a B.

4. Passe do preview à operação

Execute os capítulos de produção com $production-operations. Use migração compatível, uma revisão identificada, canário quando o risco justificar e uma prova através do domínio real. Registre latência, taxa de erro, isolamento e estado durável relevantes ao fluxo.

MCP / CONTRATO E TRANSPORTE

Exponha uma consulta a recibos

Laboratório E. Um cliente com suporte ao Model Context Protocol usa a ferramenta list_receipts. A ferramenta retorna somente recibos visíveis ao principal e usa a mesma regra de autorização da API. A primeira mudança é de leitura.

1. Valide o cliente e a versão

Com $technical-research, escolha cliente, revisão do protocolo, SDK e transporte com base no suporte demonstrável. Inicie em stdio com um subprocesso local, stdout reservado ao protocolo e logs no stderr. Escopo de arquivos, ambiente e acesso à rede pertence ao processo. Para serviço remoto, faça uma change própria para Streamable HTTP e autenticação.

2. Faça uma descoberta mínima

Instale o SDK escolhido com versão registrada. Registre no cliente de teste o executável local e seus argumentos, sem segredos. Rode com um catálogo sintético e exercite descoberta e uma chamada list_receipts antes de integrar a API. Use o cliente ou o MCP Inspector compatível com a revisão escolhida. Grave no PROOF cliente, SDK, revisão, transporte e transcript sanitizado.

3. Defina expose-tenant-receipts

- Proposta. Consultar recibos da identidade efetiva. Em stdio, vincule o subprocesso a um perfil de teste explícito. Em HTTP, derive a identidade de credencial validada. Nenhuma operação de escrita nesta change.
- Design. Reuse o serviço de consulta e aplique autorização no acesso ao recurso. Defina pageSize de 1 a 20, cursor opaco vinculado à consulta e ao principal, limite de execução e erros de argumento e serviço. Filtre dados e segredos no resultado.
- Tarefas. T1 descobre a ferramenta e lê como A. T2 verifica parâmetros, cursor de B e timeout. T3 testa cliente e transporte reais com fixture isolada. T4 revisa o pacote e a configuração de launch.

```
$mcp-server
Implement the accepted expose-tenant-receipts change only.
Use the SDK and supported transport recorded in PROJECT.
Verify discovery, bounded results, rejected input,
and denied access at the downstream service boundary.
```

MCP · transporte da revisão 2025-11-25

Teknolista · escolha por revisão e cliente

MCP / VERIFICAÇÃO E OPERAÇÃO

Teste pelo cliente que vai usar

Salve o exemplo em specs/receipt-tools/spec.md. Faça a aprovação do contrato antes de habilitar o acesso real. $behavior-testing cobre a execução comportamental e $cybersec revisa os limites de identidade, conteúdo e efeitos.

```
## Purpose

List receipts visible to the authenticated tool caller.

## ADDED Requirements

### Requirement: List authorized receipts with bounded results
The list_receipts tool SHALL return only receipts visible to the
caller, at most the requested valid page size up to twenty.

#### Scenario: Authorized client retrieves a bounded page
- **WHEN** a client for tenant A calls list_receipts with pageSize 2
- **THEN** at most two receipts are returned, all visible to A

#### Scenario: Replayed cursor crosses tenants
- **WHEN** tenant B calls list_receipts with a cursor issued to A
- **THEN** the request is rejected without returning any A data

#### Scenario: Invalid page size is rejected before lookup
- **WHEN** a client calls list_receipts with pageSize 21
- **THEN** the tool reports invalid input and performs no lookup
```

4. Rode a matriz de revisão

- Valide o delta com openspec validate expose-tenant-receipts --strict. Execute descoberta e chamada real, parâmetro inválido, timeout e cursor cruzado com A e B. Confirme no serviço e no resultado os efeitos observados. Em 2025-11-25, exercite initialize e notifications/initialized. Para outro contrato, use o ciclo exigido pela revisão adotada.
- Para HTTP, valide escopo, audiência e emissor conforme o fluxo aplicável. Exercite Origem, sessão ou handle replay por outro principal, limites do proxy, cancelamento e descoberta no cliente. Uma anotação de ferramenta orienta o cliente. A política no servidor decide se a operação pode ocorrer.

5. Evolua sem perder controle dos efeitos

Para uma ferramenta de escrita, especifique confirmação humana quando exigida, efeitos, idempotência e resposta perdida. Prove o número de registros antes e depois do retry. Para MCP remoto, aplique $production-operations e os capítulos de produção. Registre revisão, transporte, cliente e chamada crítica pelo caminho que atende os usuários.

MCP / COMPATIBILIDADE E SEGURANÇA

Verifique cada cliente suportado

Fixe cliente, SDK, revisão do protocolo, transporte, autenticação e capacidades em PROJECT. Se mais de uma combinação for necessária, mantenha uma linha de evidência por combinação.

Compatibilidade de transporte

- **2025-11-25.** Teste initialize e notifications/initialized. Em HTTP com sessão, valide credenciais em cada pedido protegido e vincule a sessão ao principal. Tente usar o mesmo identificador sem credencial e como outro principal.
- **2026-07-28.** Teste server/discover e uma primeira chamada direta válida. Confira versão e capacidades na solicitação. Em Streamable HTTP, compare versão, método e nome quando aplicável entre os cabeçalhos e a mensagem. Exercite a resposta a revisão incompatível e a rejeição antes do efeito para valores divergentes.
- **Endpoint com as duas revisões.** Faça passar o fluxo legado e o moderno. Quando os dois caminhos escrevem no mesmo serviço, prove a mesma regra de deduplicação e observe a contagem de efeitos. Diferencie o ID do protocolo da chave da operação de negócio.

Fronteiras de identidade

Em cliente local, limite o subprocesso e preserve stdout para o protocolo. Em HTTP, use HTTPS e o fluxo de autorização aplicável, valide emissor e audiência no limite correspondente e mantenha credenciais de entrada fora das chamadas a serviços de destino. No serviço, autorize recurso, ação e destino.

Trate descrições, parâmetros e conteúdo externo como entrada. Teste handle ou cursor de A usado por B, saída limitada, cancelamento e efeitos de repetição. Cache de dados privados precisa respeitar o principal e a política da revisão selecionada. Se a ferramenta escreve, simule a perda da resposta depois do commit e prove um único efeito durável.

Exercite a fixture publicada da coleção

```
python3 -m venv .venv-mcp
. .venv-mcp/bin/activate
python -m pip install 'mcp==2.2.0'
python -I -B evals/check_mcp_sdk.py
```

Execute em um checkout da coleção com ambiente de teste. A fixture usa ASGI no mesmo processo, dados em memória e identidades sintéticas. Seus resultados deixam OAuth real, persistência, streaming e infraestrutura para a matriz do produto.

MCP · mudanças da revisão 2026-07-28

Teknolista · fixture e limites

WORKER / ENTREGA ASSÍNCRONA

Recupere um job interrompido

Laboratório F. Uma exportação produz um arquivo para o solicitante. Comece com fila e armazenamento descartáveis. Exercite a repetição pelo protocolo escolhido.

1. Modele estados e efeitos

Crie generate-receipt-export e salve o delta em specs/receipt-exports/spec.md dentro da change. Em PRODUCT, defina quem solicita, quem baixa, retenção e cancelamento. Em PROJECT, registre identificador, estados queued/running/ready/failed, ownership e fronteiras transacionais. Defina o comportamento de cancelamento quando o job já tiver publicado o resultado. PROOF pede contagem de objetos visíveis e leitura pelo principal autorizado.

```
## Purpose

Publish authorized export results reliably after repeated delivery.

## ADDED Requirements

### Requirement: Publish one result for an export request
The worker SHALL expose at most one completed export per request.

#### Scenario: Worker restarts before acknowledging delivery
- **WHEN** delivery repeats after the result was committed
- **THEN** the same result is reachable and no extra export is exposed

#### Scenario: Another principal cannot fetch the export
- **WHEN** another principal requests the completed export by ID
- **THEN** no export content is returned
```

2. Construa por fatias

- T1 recebe pedido válido e cria status consultável. T2 processa uma mensagem, grava o arquivo e só anuncia ready após publicação recuperável. T3 retoma após falha, limita retries e trata mensagens inválidas. T4 exercita autorização e retenção. Use $software-delivery, $http-api para o endpoint e $behavior-testing para worker e fronteiras.
- Derive destino pelo principal autorizado. Use ID estável, registro de conclusão e operação idempotente. Se a fila e a base não compartilharem transação, explicite a estratégia para publicação pendente e reconciliação. Em ambiente de teste, interrompa antes e depois do commit e confirme recuperação.

3. Opere com um roteiro de falha

Valide com openspec validate generate-receipt-export --strict. Use $production-operations para alertar atraso e idade da fila, falhas e mensagens em quarentena. Documente como pausar consumo, descobrir a revisão, reprocessar só jobs elegíveis e confirmar contagem e autorização. Prove reexecução com um fixture antes de aplicar em produção.

SISTEMA COMPOSTO / COMPATIBILIDADE

Entregue web, API e clientes juntos

Exemplo de integração. O portal web e os clientes instalados leem recibos da API. Um servidor MCP reutiliza a regra de acesso e um worker gera exportações. A equipe precisa manter os clientes antigos durante a atualização.

Planeje a ordem pelo contrato

- **1.** Primeiro, inventarie as versões reais e o que cada cliente consome. Com $penta-pro, decida o período de compatibilidade e o significado de resultado ausente, acesso negado e falha temporária. Vincule OpenAPI, schemas MCP e requisitos OpenSpec.
- **2.** Com $software-delivery, divida por fronteira. Expanda o schema e publique a API compatível antes de promover consumidores novos. Adicione worker e MCP por perfil de teste. Valide os clientes novos e antigos contra a mesma candidata.
- **3.** Com $production-operations, faça a promoção autorizada por estágio. Anote artefato, target, migração, revisão servida, janela, gatilho de parada e recuperação. Remova o caminho antigo após confirmar que os clientes e dados relevantes já migraram.

Prove uma jornada com fronteiras reais

- Crie um recibo como A, leia pelo portal e pelo cliente MCP de A e solicite a exportação. Simule falha do worker após o commit e antes do ACK. A repetição deve produzir um único resultado visível.
- Use o cliente antigo e a configuração suportada após atualizar a API. Com B, tente o ID, o cursor e a exportação de A. Inspecione respostas, acessos e efeitos no armazenamento.
- Confirme a versão em serviço pelo caminho de tráfego normal. Observe a janela acordada e exercite a recuperação em homologação, incluindo o caso em que o código anterior não consegue ler o schema atual.

Ao responder a um incidente

Preserve a evidência e use $diagnose-defect para reproduzir com um controle funcional. Em paralelo, o responsável pode conter o incidente conforme autoridade e procedimento de produção. Depois, confirme recuperação por uma jornada real e converta a falha escapada em um cenário de regressão no dono do contrato.

Migração, promoção e recuperação

OUTROS ALVOS / PORTABILIDADE

Transfira o método para outra stack

O contrato, a divisão em fatias e a evidência sobrevivem à troca de tecnologia. A primeira etapa é identificar o consumidor, o meio de distribuição e o estado que precisa continuar válido.

CLI ou biblioteca

Exemplo de change export-notes-csv. Defina argumentos, schema CSV, encoding, stdout, stderr e códigos de saída. Em um caso inválido, a CLI deve falhar e preservar o arquivo anterior. Rode o binário empacotado via subprocesso com arquivos temporários e confira códigos e bytes. Para biblioteca, rode um consumidor das versões suportadas e teste estabilidade de API/ABI. Use $behavior-testing e $peer-review.

Dados e integração gerada

Exemplo backfill-receipt-owner. Defina pré-condições, schema, volume, batches, concorrência, reexecução e restauração. Use dry run com base sintética, conte registros antes e depois e interrompa um lote para provar retomada. Para SDK ou código gerado, edite a fonte governante e regenere. Rode ao menos um consumidor com o pacote candidato. Use $production-operations para execução autorizada em dados hospedados.

iPhone e iPad

Se o objetivo for uma experiência nativa, avalie Swift, SwiftUI e Xcode em macOS. Repita uma jornada do contrato de domínio, mas defina persistência, backup e distribuição para iOS. Exercite simulador, iPhone real, Dynamic Type e VoiceOver. Valide a candidata e o upgrade no canal de teste escolhido, como TestFlight. Use as skills genéricas e evidência da plataforma.

Embarcado ou edge

Exemplo sensor-retains-samples. Especifique unidade, relógio, limites de energia, tamanho de buffer, conexão intermitente, efeitos de reset e identidade do firmware. Use fixture de sinal e teste de bancada com hardware, perda de energia e upgrade. Documente recuperação, rollback quando seguro e critério de falha. Use $technical-research para as capacidades do dispositivo e $diagnose-defect para reproduzir sinais ambíguos.

Sistema com vários componentes

Congele interfaces entre times e descreva versão mínima suportada de cada lado. Exemplo de sequência: API compatível, web em preview, Android e desktop em canal interno, MCP habilitado para um cliente controlado. Exercite uma jornada ponta a ponta com revisões identificadas e teste a coexistência de clientes antigos e novos.

LABORATÓRIO G / CONTRATO

Da especificação a uma CLI executável

Construa uma ferramenta local que soma valores de recibos em centavos. O laboratório usa Python 3.12, biblioteca padrão, um diretório descartável e o OpenSpec do guia. As páginas seguintes contêm todos os arquivos essenciais.

Prepare o diretório e o recorte

Na raiz de um projeto novo chamado receipts-cli, inicialize Git e OpenSpec conforme a página 5. Instale PentaPRO, Software Delivery, Behavior Testing e Peer Review para exercitar o fluxo com o agente. Use Python 3.12 ou compatível e confira python3 --version. Um CSV de laboratório tem uma coluna amount_cents com inteiros decimais de 1 a 12 dígitos e cabeçalho obrigatório.

```
openspec new change sum-receipts --schema spec-driven
openspec instructions proposal --change sum-receipts
```

Proposta, design e tarefas em sum-receipts/

Salve o recorte abaixo como proposal.md. Use o delta da página seguinte em specs/receipt-totals/spec.md. O projeto é novo e essa é sua primeira capacidade.

```
## Why
Operators need a local total for a small receipt CSV.

## What Changes
- Read UTF-8 receipt amounts and print their sum in cents.
- Reject invalid files without printing a partial total.

## Capabilities

### New Capabilities
- receipt-totals: Sum receipt amounts from a local CSV.

### Modified Capabilities

## Impact
- Adds a Python CLI and subprocess acceptance checks.
```

Em design.md, registre que receipts.py usa csv com parsing estrito, soma inteiros sem conversão para float e escreve a saída somente depois de ler e validar todo o arquivo. O formato aceita arquivo somente com cabeçalho e retorna zero. O escopo cobre arquivos locais de laboratório e exclui rede, persistência e edição dos CSVs.

Em tasks.md, use o checklist da próxima página. Confirme a intenção de negócio e a autorização antes de aplicar. O script pode servir de referência para comparar com a versão produzida pelo agente.

LABORATÓRIO G / DELTA E TAREFAS

Combine cenário, tarefa e verificação

Salve o delta em openspec/changes/sum-receipts/specs/receipt-totals/spec.md. Os títulos permitem vincular os testes do CLI aos cenários aceitos.

```
## Purpose

Compute a validated total in cents from a local receipt CSV.

## ADDED Requirements

### Requirement: Total valid receipt amounts
The CLI SHALL read a UTF-8 CSV with one amount_cents column.
It SHALL accept amounts with one to twelve ASCII decimal digits.
It SHALL print the integer sum and a newline on success.

#### Scenario: Valid amounts are summed exactly
- **WHEN** the file contains 1250 and 75 cents
- **THEN** stdout is "1325" plus a newline and exit code is 0

#### Scenario: A header-only file returns zero
- **WHEN** a valid file contains the header and no data rows
- **THEN** stdout is "0" plus a newline and exit code is 0

### Requirement: Reject invalid or unavailable input
The CLI SHALL return exit code 2 and an error on stderr for an
invalid or unreadable file, with empty stdout.

#### Scenario: Invalid row after a valid amount
- **WHEN** a valid amount is followed by a negative value
- **THEN** the CLI reports an error and emits no partial total
```

Salve em tasks.md

```
## 1. Deliver receipt totals
- [ ] 1.1 Read and validate the CSV and return the exact sum.
- [ ] 1.2 Verify success and failure through the CLI entry point.
- [ ] 1.3 Review the revision and record the evidence.
```

```
openspec status --change sum-receipts
openspec validate sum-receipts --type change --strict
openspec instructions apply --change sum-receipts --json
```

A validação confirma o formato. Leia as regras e confira se os testes vão observar o código de saída e os dois streams. Aplique a change só no escopo acordado.

LABORATÓRIO G / IMPLEMENTAÇÃO

Implemente o ponto de entrada

Use $openspec-apply-change e $software-delivery no escopo aceito. Peça a implementação pelo caminho real de execução. Se preferir praticar sem agente, salve a referência abaixo como receipts.py na raiz.

```
import argparse
import csv
import re
import sys
from pathlib import Path


def total_cents(path: Path) -> int:
    with path.open(encoding="utf-8", newline="") as source:
        rows = csv.reader(source, strict=True)
        if next(rows, None) != ["amount_cents"]:
            raise ValueError("invalid header")
        total = 0
        for row in rows:
            if len(row) != 1 or not re.fullmatch(r"[0-9]{1,12}", row[0]):
                raise ValueError("invalid amount")
            total += int(row[0])
        return total


def main() -> int:
    parser = argparse.ArgumentParser(description="Sum receipt cents")
    parser.add_argument("csv_file", type=Path)
    args = parser.parse_args()
    try:
        result = total_cents(args.csv_file)
    except (OSError, UnicodeError, ValueError, csv.Error):
        print("Invalid or unreadable receipt CSV", file=sys.stderr)
        return 2
    print(result)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Inspecione o desenho

O efeito observável ocorre em stdout depois da validação integral. O erro usa stderr e preserva o arquivo. A soma usa inteiros. Para um produto que recebe arquivos de terceiros, especifique também os limites de entrada, volume, tempo, recursos e caminhos autorizados.

LABORATÓRIO G / TESTE E ARQUIVO

Prove a saída real e feche a change

Use $behavior-testing ou execute o teste de referência na raiz. O subprocesso exercita o mesmo ponto de entrada usado no terminal. Os arquivos temporários são descartados ao fim da execução.

```
python3 - <<'PY'
from pathlib import Path
import subprocess
import sys
import tempfile

with tempfile.TemporaryDirectory() as tmp:
    fixture = Path(tmp) / "receipts.csv"
    cases = [
        (b"amount_cents\n1250\n75\n", 0, "1325\n"),
        (b"amount_cents\n", 0, "0\n"),
        (b"amount_cents\n1250\n-1\n", 2, ""),
        (b"amount_cents\n\"1250", 2, ""),
        (b"amount_cents\n\xff\n", 2, ""),
        (None, 2, ""),
    ]
    for payload, status, stdout in cases:
        if payload is None:
            fixture.unlink()
        else:
            fixture.write_bytes(payload)
        run = subprocess.run(
            [sys.executable, "receipts.py", str(fixture)],
            capture_output=True, text=True, timeout=5,
        )
        assert (run.returncode, run.stdout) == (status, stdout), run
        assert bool(run.stderr) == (status != 0), run
        if payload is not None:
            assert fixture.read_bytes() == payload
print(f"{len(cases)} CLI cases passed")
PY
```

Em uma cópia isolada, retire a validação de valores e repita o caso negativo. Ele deve falhar. Restaure a implementação e rode todos os checks. Revise o diff e registre comando, revisão, casos e resultados. Marque as tarefas com evidência.

```
openspec archive sum-receipts
openspec list --specs
openspec validate --all --strict
```

Arquive após concluir os checks e confirmar a autoridade. Confira receipt-totals e seu histórico. Na 1.17.0, um Purpose provisório pode reprovar a validação estrita. Complete a finalidade na spec consolidada se necessário.

11 / PILOTO

Retome com contexto verificável

Uma sessão nova deve conseguir descobrir a revisão, o contrato, o trabalho pendente e a próxima checagem pelos arquivos do projeto.

DOCS/PROCESS.MD / PASSAGEM DE CONTEXTO

```
- Base and candidate revision: <refs or hashes>
- Accepted change and scope: <ID and scenarios>
- Completed work: <task IDs and evidence links>
- Pending or blocked: <check, owner, reason>
- Next authorized action: <one increment or check>
- Toolchain and test target: <versions, variant, API, ABI>
- Decisions since last session: <links or none>
```

CHAT / RETOMADA EM UMA SESSÃO NOVA

```
$software-delivery
Read AGENTS.md, docs/process.md, and the active OpenSpec change.
Confirm the current branch and working tree before editing.
Use the latest accepted contract and revision-specific evidence.
Run the next authorized check or task. Report any drift.
```

01 **Teste o handoff no piloto.** Encerre uma sessão entre as duas fatias e comece outra com os documentos. Registre tempo até a primeira ação correta e quais informações precisaram ser repetidas.

02 **Reabra a evidência se houver mudança.** Reveja o diff, requisitos, resultados afetados e autorizações. Preserve os resultados anteriores como histórico da revisão que os produziu.

Anthropic: contexto de retomada em agentes de longa duração

11 / PILOTO

Defina o ensaio antes de começar

O plano da Teknolista propõe preparar controles, testar melhorias pequenas e só então avaliá-las em um produto real.

Etapa	O que fazer no próximo projeto	O que permite avançar
P0 / Preparar	Fixe a revisão da coleção, CLI, modelo, permissões, caso e critérios. Use dados fictícios e um controle com defeito e um limpo.	A checagem falha no defeito conhecido e passa no caso bom. O avaliador tem acesso separado.
P1 / Exercitar	Faça uma edição de texto direta, a primeira change OpenSpec e uma retomada em sessão nova. Inclua promoção de protótipo se ela fizer parte do projeto.	O fluxo direto continua curto. A change tem evidência por cenário. A retomada usa o contrato sem rediscutir decisões.
P2 / Pilotar	Use uma mudança real com risco de dados, como fixar notas e migrar a base. Adicione revisão independente consultiva e uma distribuição controlada.	Você pode comparar qualidade, esforço, revisão e retrabalho. Há evidência de upgrade e recuperação.

01 **Fixe os critérios.** Por exemplo, a variante precisa preservar notas v1 e rejeitar dados inválidos. Defina também custo e tempo aceitáveis antes de examinar os resultados.

02 **Use controles sem dados reais.** Um revisor deve achar a migração que apaga uma nota sem inventar problema em um caso limpo. Uma solicitação contraditória em log deve ser tratada como conteúdo do teste.

COMO COMEÇAR PEQUENO

Execute manualmente os dois incrementos e a retomada. O executor automatizado da coleção tem fixtures próprias. Use-o quando quiser comparar versões de skills em condições controladas.

Plano de adoção Teknolista

11 / PILOTO

Compare duas formas de trabalhar

Compare revisões de skills com o mesmo caso. Avalie o efeito de revisão independente em um ensaio próprio, para identificar o que mudou.

01 **Escolha A e B.** A usa a revisão atual. B muda uma instrução, skill ou checagem específica. Use cópias separadas do mesmo commit e os mesmos dados semeados.

02 **Mantenha as condições.** Registre modelo, host, nível de raciocínio quando aplicável, ferramentas, versões, orçamento, tempo máximo, política de autorização e matriz Android. Alterne a ordem dos ensaios.

03 **Meça o comportamento.** Use avaliador com casos esperados fora do alcance do agente. Confira a estrutura dos artefatos, diff, testes e efeitos observados. Registre ativações erradas, falsos achados e evidência incompleta.

04 **Repita os casos críticos.** Três tentativas por condição servem como triagem exploratória. Use mais casos e outro host ou modelo antes de generalizar. Mantenha execução com e sem menção explícita como condições distintas.

TERMINAL / NA COLEÇÃO, FORA DA RAIZ DO APP

```
python3 scripts/run_skill_evals.py --check-isolation
python3 scripts/run_skill_evals.py --baseline 9405717 \
  --arms baseline candidate --cases delivery-openspec review-clean
```

O segundo comando usa o modo local exploratório e requer Codex CLI autenticado e OpenSpec no PATH. Ele avalia as fixtures da coleção, incluindo um handler sintético. Para comparação cega, siga o modo isolado Linux com Bubblewrap e gateway configurado. O isolamento de mounts e processos requer política de rede própria para ensaios adversariais.

Procedimento e condições do executor de avaliação

11 / PILOTO

Registre qualidade, custo e esforço

Use uma linha por tentativa e uma janela comparável para observar defeitos após a integração.

Métrica	Como registrar
Aceitação dos cenários	Passaram / total exigido. Registre quais falharam e quais foram bloqueados.
Falhas relevantes detectadas	Achados válidos sobre defeitos conhecidos. Registre também omissões, alarmes falsos e execução fora de escopo.
Tempo e custo	Minutos até primeira evidência, revisão humana, retrabalho e consumo ou custo do agente quando medível.
Retomada	Tempo até ação correta e contexto que precisou ser fornecido de novo.
Após a integração	Regressões, reverts e falhas reportadas dentro da janela escolhida. Relacione à revisão quando possível.

REGISTRO DO PILOTO / UMA LINHA POR EXECUÇÃO

```
Case | Arm | Attempt | Commit | Skill revision | Model
Device/API | Toolchain | Checks passed/required | Missed defects
False findings | Human minutes | Rework | Token/cost | Decision
```

CRITÉRIO DE ADOÇÃO

Adote uma mudança no fluxo quando o benefício se repetir no caso-alvo com custo aceitável e sem regressão nos controles. Trate um efeito não autorizado ou uma alegação de verificação sem prova como falha da rodada.

11 / PILOTO

Adapte as práticas da Anthropic

As publicações mostram exemplos de equipes, recomendações e experimentos. Use o efeito que você quer medir para escolher o que adaptar.

Prática documentada	Aplicação no piloto	Sinal para manter
Progresso persistido e retomada com contexto novo.	PROCESS referencia change, revisão, pendências e próximo passo.	Menos tempo para retomar, com menos contexto reexplicado.
Revisão em camadas e atenção humana nos riscos.	Checagem local primeiro, revisão independente consultiva em migração, decisão humana de integrar.	Mais defeitos materiais capturados sem retrabalho desproporcional.
Critérios executáveis e controles de avaliação.	Cenários OpenSpec mapeados a testes de Android e caso defeituoso controlado.	Resultado de avaliação se mantém ao repetir em condições iguais.
Divisão entre planejamento, geração e avaliação em experimentos.	Em uma change de maior risco, separe quem define aceitação, quem implementa e quem avalia.	A segunda leitura acha uma falha que muda a decisão.

O uso interno de OpenSpec e a adoção geral de SDD pela Anthropic permanecem sem confirmação pública. A combinação com PentaPRO e OpenSpec é uma escolha da Teknolista para este piloto.

ONDE FAZER A CORREÇÃO

Falha específica deste app vai para o teste ou AGENTS.md do app. Uma instrução reutilizável entra na skill quando o problema se repetir em contextos distintos. Uma regra objetiva que deve bloquear integração vai para a CI após validar seus controles.

Playbook público de SDLC com IA

Revisão no time do Claude Code

Avaliações de agentes

11 / PILOTO

Feche a retrospectiva com decisões

Separe resultados observados de hipóteses. Para cada melhoria, escreva o problema recorrente, a menor intervenção e o próximo teste.

Se o piloto mostrar	Próxima ação
Nota desaparece só no upgrade	Reproduza com banco antigo e crie teste de migração. Revise a checagem obrigatória para mudanças de schema.
Revisor cria ruído em mudanças pequenas	Meça falsos achados em um caso limpo. Mantenha a revisão independente nos riscos com ganho demonstrado.
Agente reabre decisões já aceitas	Ajuste o apontador no PROCESS ou o briefing. Teste a retomada em sessão nova.
Check verde examina revisão errada	Vincule resultado ao commit, variante e versionCode observado. Use um controle com revisão antiga servida.
A skill pede ferramentas que o host não possui	Documente pré-requisitos, caminhos alternativos e limites. Teste a versão revisada no ambiente real.

CHAT / RETROSPECTIVA BASEADA NAS EVIDÊNCIAS

```
$agent-instructions
Review the pilot evidence and recurring failures.
Propose one small change to instructions, tests, or skill routing.
Name its owner and a positive and negative control.
Separate observed gains, regressions, and open hypotheses.
```

DECISÃO A REGISTRAR

Adotar, ajustar ou adiar. Diga qual evidência justificou a escolha, quem é o responsável, qual revisão será avaliada e o que caracteriza sucesso na próxima rodada.

APÊNDICE A / CATÁLOGO

Skills gerais e de entrega

No Codex, use $ seguido do identificador ou selecione a skill no host. Versões da coleção local desta edição.

Identificador	Versão	Use para
penta-pro	2.8.0	Entrevista, especificação, cinco pilares e prontidão.
software-delivery	1.4.0	Coordenação de implementação e verificação integrada.
production-operations	1.0.0	Infraestrutura, pipeline, deploy, verificação e recuperação.
ui-design	1.5.7	Telas, estados, protótipos e acessibilidade.
technical-research	1.0.0	Pesquisa técnica e experimento delimitado.
diagnose-defect	1.0.0	Reprodução, hipótese e evidência de causa.
behavior-testing	1.1.0	Testes de comportamento e fluxo test-first.
http-api	1.1.1	Endpoints, autorização, integração e efeitos HTTP.
mcp-server	1.1.0	Servidor, protocolo, ferramentas e efeitos MCP.
peer-review	1.10.2	Revisão de código, testes e evidência.
cybersec	1.6.0	Ameaças, controles, auditoria e hardening.
git-delivery	1.0.0	Commit, push e PR conforme o escopo autorizado.
agent-instructions	1.1.0	Skills, AGENTS.md e avaliação.
pr-quiz	1.3.0	Quiz de compreensão de PR quando solicitado.

As oito skills de Android nativo estão na página 101. Escolha pelo resultado pedido e combine as skills quando suas tarefas se encadearem.

APÊNDICE B / CATÁLOGO

Helpers de Android nativo

Escolha pelo trabalho concreto. O projeto e seus contratos determinam dependências, bibliotecas, versões e estrutura.

Identificador	Versão	Use para
android-module-structure	1.3.0	Gradle, fronteiras de módulos e convenções nativas.
android-data-layer	1.2.4	Repositórios, rede, Room, DataStore e sincronização.
android-error-handling	1.2.3	Resultados e erros tipados nas fronteiras Kotlin.
android-di-koin	1.2.3	Bindings, resolução de dependências e ciclo de vida.
android-background	1.2.3	Trabalho durável, retries e restrições WorkManager.
android-unit-testing	1.2.3	Testes isolados de lógica, estado e corrotinas.
android-integration-testing	1.2.4	Testes de fronteiras de dados, DI e jobs.
android-ui-testing	1.2.3	Testes de interação Compose em emulador ou dispositivo.

Exemplo de seleção para o laboratório

Defina telas com UI Design quando estiver especificando a interface. Use Android Data Layer para a persistência. Teste estado e validação com Android Unit Testing, a fronteira de gravação com Android Integration Testing e a interação com Android UI Testing. Para uma migração real, use uma base antiga com dados e confira o estado depois do upgrade.

O fluxo com OpenSpec no Codex

Core: $openspec-explore, $openspec-propose, $openspec-apply-change, $openspec-update-change, $openspec-sync-specs e $openspec-archive-change.

Expandido: selecione os workflows desejados em `openspec config profile` e aplique com `openspec update`. Use os identificadores gerados para new, continue, ff, verify, bulk-archive e onboard conforme a tarefa e a autorização.

REFERÊNCIA RÁPIDA

Consulte a lista de skills no host e as pastas **.agents/skills/**. O seletor, o registro de atividade e os artefatos observados ajudam a separar disponibilidade, seleção e aplicação.

APÊNDICE C / EVIDÊNCIA DA EDIÇÃO

Saiba o que foi verificado

Os resultados abaixo pertencem a escopos diferentes. Registre seus próprios resultados por versão do produto, cenário, ambiente e canal.

Coleção de skills

A revisão da coleção sobre 640d5ed validou as 22 skills, 87 testes offline e 818 links ou âncoras locais. Skills CLI 1.7.6 e instalação manual reproduziram 144 arquivos de pacote. O Codex CLI 0.160.0 descobriu as 22 skills no projeto de teste. Esses resultados cobrem a instalação e as checagens executadas.

Exemplos deste guia

O laboratório CLI executou seis casos com Python 3.12.6. O teste detectou uma versão que aceitava valores negativos. O OpenSpec 1.17.0 validou os nove deltas de exemplo e o arquivo da CLI, incluindo sua spec consolidada. No upgrade Android, retirar um cenário preservado fez a validação falhar. Execute a verificação de comportamento no alvo de cada capítulo.

O SDK MCP 2.2.0 passou nos cenários locais documentados de descoberta, duas revisões, isolamento de caller e repetição após resposta perdida. O ensaio é sintético. O capítulo MCP explica quais verificações executar no produto.

Antes de avançar

- **Implementar.** Defina escopo, comportamento, falha representativa e decisão de autoridade. Execute o preflight que o alvo requer.
- **Arquivar.** Concilie tarefas, delta e prova da revisão candidata. Inspecione a spec consolidada e valide.
- **Publicar e promover.** Verifique diff, destino, políticas, artefato, identidade, compatibilidade e recuperação. Confirme o efeito pelo canal real.
- **Encerrar.** Registre revisão em uso, resultados, lacunas e próximo dono. Mantenha evidências históricas datadas.

Checklist por alvo
Catálogo e instalação da coleção

APÊNDICE / CHECKLIST

Confirme o resultado no alvo

Use a checklist comum de prontidão e entrega deste apêndice e aplique a checagem abaixo ao canal e à plataforma. Dê a cada pendência um dono e uma próxima ação.

- **Primeira tarefa.** Um clone limpo reproduz a instalação. O ambiente abre o alvo. Escopo e autoridade estão explícitos. A mudança material tem contrato e cenários vinculados ao plano.
- **Antes de concluir.** O resultado passa na entrada real. Um caso negativo representativo detecta falha. Revisão, comando, alvo e efeito foram registrados. O próximo leitor consegue encontrar o contrato e a prova.
- **Android e desktop.** Identidade, assinatura, variante, hash, versão, canal e dispositivo estão ligados à evidência. Prove instalação e upgrade com dados antigos, acessibilidade e recuperação no alvo.
- **Web.** Confirme versão servida, rotas, sessão e teclado no domínio e navegador contratados. Teste a compatibilidade com a API e uma recuperação após falha.
- **HTTP e MCP.** Confirme versão, identidade permitida e negada, resposta e efeito persistido no cliente/protocolo adotado. Revise retries, carga e parada de rollout conforme o risco.
- **Dados e workers.** Confirme backup restaurável, lote repetível, progresso observável, integridade e responsável pela retomada.
- **Mudanças de skill.** Use $agent-instructions, com caso limpo, defeito reproduzível e critérios antes do ensaio. Registre se deve adotar, ajustar ou adiar. Use $pr-quiz quando houver pedido de compreensão das decisões do PR.

Pacote mínimo para passar o trabalho

```
Revision and artifact: <commit, version, variant, digest>
Accepted contract: <change, capability, scenario>
Observed: <command, environment, output, durable effect>
Pending: <check, limit, owner, next action>
Delivery: <target, authority, current rollout, recovery>
```

12 / APÊNDICE

Resolva os bloqueios comuns

Comece pela primeira falha reproduzível e anote em qual camada ela ocorre.

Sintoma	Primeiro passo
Gradle funciona no Studio e falha no terminal	Compare ./gradlew --version com o JDK escolhido no Studio. Confira o SDK localizado e a compatibilidade do wrapper e AGP.
OpenSpec não acha a change	Confira a raiz do app, openspec list, ID, perfil e schema. Se já foi arquivada, use a spec consolidada e o histórico.
Validação do delta falha	Abra a instrução do artefato. Confira cabeçalho, capacidade e cenário. Em MODIFIED, preserve os cenários anteriores que continuam válidos.
Instrumentação sem dispositivos	Inicie e desbloqueie um AVD. Verifique adb devices -l e o alvo permitido. Confirme que a variante tem testes.
Reabertura sempre passa no fake	Repita a jornada com Room e finalize o processo. Confira que a segunda abertura lê dados persistidos.
Upgrade perde dados ou recusa instalar	Inspecione identidade, assinatura e versionCode. Confira a migração, schema exportado, defaults e política para bases antigas.
AAB aprovado, app antigo no telefone	Confira adesão à trilha, conta, disponibilidade e versionCode instalado. Registre a revisão que realmente executou.

SEGUIR COM SEGURANÇA

Traga a causa, o comando, o alvo e a saída mínima para a próxima sessão. Use diagnose-defect para investigar problemas sem causa estabelecida.

12 / APÊNDICE

Glossário para a primeira leitura

Use estes termos para entender os comandos, as etapas e a evidência esperada.

Termo	Significado no guia
Schema do OpenSpec	Define quais artefatos uma change precisa e como eles se relacionam.
Delta de spec	Requisitos e cenários adicionados, modificados ou removidos por uma change.
SDK, API e ABI	Ferramentas e bibliotecas Android, nível da plataforma e arquitetura binária do dispositivo, respectivamente.
AGP e wrapper	Android Gradle Plugin e mecanismo do repositório que fixa a distribuição do Gradle.
minSdk / compileSdk / targetSdk	Menor API suportada, API disponível para compilar e nível de comportamento alvo do app.
AVD e ADB	Dispositivo virtual Android e ferramenta de terminal que se comunica com dispositivos.
Variante	Combinação do tipo de build, como debug ou release, e flavors quando houver.
Fake e instrumentação	Dependência de teste controlável e execução de testes em um ambiente Android.
PR e CI	Pedido para integrar alterações e execução automatizada de checks.
Upgrade e migração	Instalar uma versão posterior preservando dados e transformar a estrutura existente quando ela mudou.

Se um comando usar um caminho como /caminho/v1.apk, substitua pelo arquivo real do projeto. Os textos em inglês dentro dos blocos correspondem aos artefatos ou prompts do laboratório.

12 / APÊNDICE

Referências para consultar

Use as páginas oficiais para conferir versões, opções do CLI e requisitos de distribuição antes de aplicá-los ao seu projeto.

Coleção e laboratório

Repositório Teknolista na revisão do tutorial

Guia do piloto e estado das avaliações

OpenSpec: início rápido e ciclo de uma change

OpenSpec: referência do schema spec-driven

Android e Play

Arquitetura de apps Android

Testes, ambientes e fronteiras

Backup e extração de dados

Migração de bancos Room

Assinatura e publicação

Engenharia com agentes

Anthropic: projeto de agentes para tarefas longas

Anthropic: práticas de ciclo de vida com IA

Anthropic: desenho de avaliações de agentes

BASE DESTA EDIÇÃO

No CLI 1.16.0, a primeira change validou e arquivou com conclusão simulada das tarefas. A segunda validou e rejeitou a omissão de cenário. Build, testes em dispositivo e Play ficam para o seu ambiente.

APÊNDICE / FONTES

Consulte as edições e os ensaios

A base geral corresponde ao guia de 10 de novembro. A trilha Android parte do tutorial de 11 de novembro. Os controles de processo vêm da retrospectiva do Castas de 9 de novembro. As referências a resultados mantêm a edição e a revisão em que foram observados.

Fontes da consolidação

- **Guia SDD, OpenSpec e skills Teknolista.** Fluxo comum, laboratórios A e B, produção, caminhos direto e exploratório, retomada e catálogo.
- **Tutorial Android com SDD e OpenSpec.** Base Kotlin/Compose, criação offline, Room, revisão, migração, candidata, Play e piloto. Instalação, pilares e roteamento usam a referência compartilhada.
- **Castas Android, auditoria retrospectiva.** Cronologia e lições de preflight, paridade, testes, CI, documentação, acessibilidade, privacidade, entrega e adoção. Os resultados do Castas são históricos.

Como interpretar os resultados

Os ensaios do OpenSpec, o laboratório executável e as verificações da coleção estão descritos no apêndice de evidências. Os demais capítulos e matrizes trazem exercícios e critérios para executar nos seus projetos. Antes de usar um provedor, SDK, versão de protocolo ou canal de distribuição, confira os requisitos e os recursos disponíveis na conta e no ambiente alvo.

Resultados e limites desta edição
OpenSpec · release 1.17.0
Teknolista · catálogo publicado
Teknolista · plano e métricas de avaliação
Teknolista · guias de operação
Android Developers · preparação e distribuição
MCP · especificação por revisão